

AgtXIV

Source-Grounded Scientific Records and a Publishable Research Interface

AgtXIV Project

Technical report and maintenance rules
8 September 2026

Abstract

Scientific results are reusable only when their source, assumptions, supporting argument and unresolved conditions remain accessible together. AgtXIV represents these elements as separately versioned records, with exact evidence references and six distinct assessment questions. This report describes the experimental V3 contract package, its local validation and candidate-storage mechanisms, and the architecture of a new-paper web interface. A source-grounded robustness-of-magic example illustrates why an optimal decomposition, trace preservation and postselection must remain distinct in a scientific record. Existing local evidence establishes a reproducible candidate-processing path and rejection tests for several incorrect bindings; it does not establish completed scientific review or formal knowledge admission. The report also specifies API, release and repository-maintenance boundaries. A generated appendix covers all 64 record families and 19 shared value structures in the bound contract package, so the field reference can be regenerated rather than manually synchronized.

Evidence boundary. This is an implementation-oriented working report. The project Charter remains pending ratification. V3 denotes the architecture; 0.0.0 denotes the experimental record contracts. Operational success, schema validity and scientific acceptance are different claims. The new-paper path has retained local HTTP, browser and responsive-interface evidence; public deployment and the complete repository release gate remain distinct checks.

Contents

1	Scientific purpose and reading model	3
2	Exact identity, sources and claim reconstruction	4
3	Arguments and six separate assessment questions	5
4	Record families across the lifecycle	7
5	Worked example: robustness of magic	9
6	New-paper web and API architecture	10
7	Implementation evidence, maintenance and release	14
8	Boundaries, evaluation and next work	16
A	Generated contract field reference	17

1 Scientific purpose and reading model

AgtXiv aims to make a scientific result inspectable at the level at which someone would reuse it. A reader needs more than a paper title: they need the precise proposition, its assumptions, the argument that supports it, the source passage, the checks performed and any remaining obstacle. A result about pure states, for example, does not automatically apply to mixed states. Preserving the mathematical formula while dropping that distinction changes what a reader is entitled to conclude.

The useful mental picture is a laboratory notebook attached to a collection of scientific claim cards. A source snapshot preserves the original sample. A claim card states what was asserted and under which conditions. An argument connects several cards through an explicit reasoning step. An assessment records what a particular check concluded about an exact target. The notebook retains unsuccessful attempts and competing assessments, rather than replacing them with a single colour or score.

The project’s contribution at the present checkpoint is an explicit record architecture, implemented local consistency checks and bounded candidate-processing mechanisms. It has not measured an improvement in research productivity, demonstrated general scientific verification, or completed the entire V3 lifecycle. The report therefore treats reliability claims as engineering claims about named checks and supplied records. Scientific adequacy remains a distinct question requiring evidence appropriate to the domain.

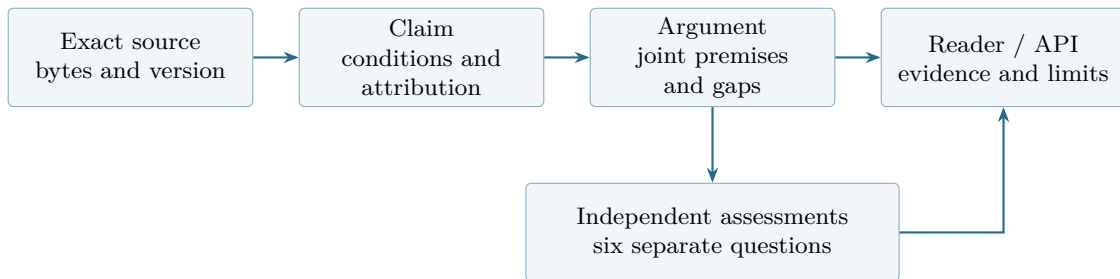


Figure 1: The reading path and evidence path meet at the interface. Arrows in this overview show information flow. They are not themselves mathematical implications. Assessment and publication decisions require their own records.

1.1 Three related products

The project produces a protocol, a working implementation and a scientific reading interface. The protocol describes records and constraints; code enforces the implemented subset; the interface helps humans and other programs inspect actual records. A publishable version requires these three products to agree. A field described in a schema does not establish that its producer exists, and an attractive interface cannot supply missing evidence.

The new-paper product path adds an operational requirement: a user submits a new arXiv identifier, the service acquires an exact source version, performs bounded analysis, reports actual task progress and exposes the resulting evidence. This path is more than a replay of a precomputed example. Its first output is a source-grounded candidate view, whose open questions remain visible. Section 6 separates this operational path from the richer V3 scientific contract.

1.2 Stable terminology

Term	Meaning in this report
Paper Agent	A versioned, source-bounded collection of records about a paper; not automatically an independently qualified reviewer.
Scientific claim	A proposition attributed to a source, preserving scientific conditions and modality.
Candidate	A proposed extraction, relation or interpretation whose scientific status has not been admitted by the required process.
Assessment	A method-specific conclusion about an exact target, with conditions and evidence.
Snapshot	An explicit version boundary defining which records are visible. Its existence does not imply approval.
Frontier item	A retained unresolved question, with the affected target and the next evidence needed.
API	Application programming interface: the contract by which a program submits work or reads records.

V3 is the architecture version, whereas 0.0.0 is the experimental contract version. Software version, API version, paper version, record revision and knowledge snapshot are independent identifiers. The proposed Charter is not currently effective; this report does not ratify it.

2 Exact identity, sources and claim reconstruction

2.1 Two hashes answer different questions

An artifact hash identifies original bytes. A record hash identifies a canonical serialized record. These questions differ: a reformatted JSON file can encode the same canonical record while having different original bytes. Conversely, unchanged paper bytes can support several different candidate interpretations. AgtXiv keeps these identities separate.

I1: exact reference. A reference to a record binds the tuple

$$r = (\text{record type}, \text{lineage identifier}, \text{revision}, \text{content hash}). \quad (1)$$

The revision is a positive integer. A floating request for the latest version is a convenience query and cannot stand in for this tuple in reproducible evidence. The same lineage and revision cannot be silently reused for changed content.

I2: record envelope. Each V3 record includes its exact discriminator, identifier, revision, creation time, material class, schema-bundle hash, attributed producer, policy reference, input references, content hash and payload. The canonical record hash excludes only the root `content_hash` field. Nested hashes and references remain covered. The implementation uses the existing V2 normalization and integer-safe JSON parser while hashing the V3 flat envelope with its own procedure.

I3: original bytes. Byte-bearing references bind an artifact identifier, media type, byte size and SHA-256. A filename is a locator, not proof of identity. The local validators reject unresolved references, stale hashes and duplicate identities inside the supplied set. Their scope does not establish that the caller supplied every relevant record in the world.

The bundle manifest binds schema bytes and the offline resolver uses local identifiers. Schema URIs are not instructions to download arbitrary network resources during validation. The generated appendix binds the source manifest used in this report and records the individual input digests in `generated/schema-inputs.json`.

2.2 Acquisition precedes interpretation

The request identifies the intended paper; acquisition records what was actually obtained. An omitted arXiv version must be resolved and then stored as an exact version for that run. A request, a successful response, a downloaded archive and a published rendering are different observations. Missing source packages, absent supplementary data and unsupported formats require explicit outcomes.

S1: source boundary. Preserve the selected source bytes and an inventory of the files actually captured. Record their origin and their relationship to the work and version. In the existing robustness example, six original/archive files and eight locally generated build files were all captured. The additional build files were observed artifacts, not evidence that this task executed a paper build.

S2: span boundary. For text artifacts, byte spans use zero-based, half-open intervals $[a, b)$ over the original bytes. A span binds the parent artifact and selected byte hash. Human-facing line numbers aid navigation but do not replace byte identity. If normalization changes text, retain a transformation record linking its input and output; do not attach normalized offsets to the original artifact.

S3: activity boundary. TeX is a programmable document format. Static inspection can identify some includes, environments and local references, but does not establish complete macro expansion or published rendering activity. Commented, inactive and unknown material must remain distinguishable. The source pipeline processes data without executing untrusted TeX or scripts.

2.3 A source claim is not a normalized theorem

A source-faithful claim retains its statement, source spans, components, conditions, modality, attribution, scientific system and comparison baseline. Mathematical normalization records typed objects, quantifiers, assumptions and conclusions in a separate representation. A component map states which parts were represented mathematically and which scientific meaning remains outside that representation.

S4: preserve meaning. Dropping a quantifier, changing a physical system, replacing an approximation with an equality or adding a new premise requires an explicit account. A harmless representation correction may remain within a lineage after suitable review; a changed proposition needs a distinct claim identity or explicitly derived claim. Neither a text similarity score nor a matching hash is an oracle for semantic identity.

S5: freeze the denominator. An inventory records discovered claims, non-claim material and unknown content. A frozen scope binds a reviewed inventory and the required work. A later failure cannot be hidden by deleting an obligation. Corrections to omissions, additions of work and new scientific hypotheses follow different revision paths, with old records retained.

Evidence boundary. A lexical candidate inventory is a reproducible reading aid. Its number of lines, spans or cues is not a count of scientifically distinct propositions, and complete byte coverage does not establish complete semantic discovery.

3 Arguments and six separate assessment questions

3.1 Joint premises and local assumptions

An argument is represented through nodes, inference steps, assumption contexts, proof routes and explicit dependency bindings. A joint inference has the form

$$(P_1, \dots, P_k; \Gamma, \mathcal{R}) \implies Q, \quad (2)$$

where Γ names the local assumptions and $\mathcal{R}$ the stated rule. This representation preserves the fact that the premises are used together. Drawing independent arrows $P_i \rightarrow Q$ without recording their joint use can incorrectly suggest that each premise alone proves the conclusion.

A1: distinguish graph meanings. Document containment, provenance, mathematical dependence and scientific comparison are different relations. A heading containing a theorem is not a proof dependency. A paper citing another paper does not by itself identify a load-bearing theorem. The interface must expose relation type and candidate status before relying on graph reachability.

A2: preserve context. A statement proved under a local assumption cannot escape that context merely because a producer labels the assumption discharged. The implemented record checks constrain supported discharge forms and evidence references, but do not themselves prove every inference rule. Alternative proof routes remain separate, so a gap in one route does not automatically negate another.

The pinned vibefeld adapter illustrates another boundary: replaying supplied ledger bytes and comparing them with an export can establish internal consistency of that capture. It does not authenticate declared identities or establish that an independent upstream process actually ran. Upstream labels such as

validated, clean, closed and admitted remain attributed reports. They are not V3 scientific approval or knowledge admission.

3.2 What the six axes ask

Axis	Scientific question	Example evidence
Source fidelity	Does the recorded proposition preserve what the source says, including attribution and conditions?	Exact spans and source review
Mathematical correctness	Does the stated mathematical conclusion follow under the recorded assumptions?	Argument review or suitable formal evidence
Formal alignment	Does the formal statement express the intended mathematical target?	Exact endpoint comparison and backtranslation
Semantic applicability	Do mathematical objects, assumptions and approximations apply to the scientific use under discussion?	Model and domain review
Empirical support	Do observations support the scientific claim under the stated methodology and uncertainty?	Data and empirical review
Computational reproducibility	Can the specified computation be reproduced under its input, environment and comparison protocol?	Execution records and reproduction review

A3: no automatic transfer. Lean kernel success concerns a formal declaration in an environment. It does not, alone, establish source fidelity, correct translation, physical applicability or empirical support. Similarly, reproducing a plotted numerical value does not establish that the experiment supports the physical interpretation assigned to it.

3.3 Applicability, conclusion and execution

Each axis assessment separates three state dimensions. Applicability asks whether the question applies: UNDETERMINED, APPLICABLE or NOT_APPLICABLE. Result records the evidence conclusion: NOT_ASSESSED, INCONCLUSIVE, PARTIALLY_SUPPORTED, SUPPORTED or COUNTEREVIDENCE. Execution records what work occurred: DEFERRED, BLOCKED, FAILED or COMPLETED. A method, exact targets, review context, supporting and opposing evidence, conditions and unresolved frontiers accompany these labels.

A4: retain disagreement. A status view is a read-only projection over a supplied record set. It must preserve differing assessments of the same exact target, including their conditions and methods. Selecting the newest positive assessment while hiding an earlier relevant objection is not an acceptable reduction. Missing evidence is not zero evidence, and an unmeasured quantity is not a measured zero.

The local `project_axes` function retains the assessments of the exact target in the supplied collection. A serialized `StatusView` is a separate record whose validation checks implemented bindings; it is not a mechanical proof that its author supplied every assessment. Neither mechanism establishes completeness outside the submitted record universe.

A5: distinguish success from accounting. A work obligation can be accounted for by a documented failure or deferral where its processing profile permits this. An obligation marked `SUCCESS_REQUIRED` additionally needs suitable success evidence for the exact target. Filling a result-reference field with an arbitrary existing record cannot discharge that requirement. Unsupported completion stages remain fail-closed in the local implementation.

3.4 Independence is more than a different name

Review records bind who reviewed what, which inputs were visible and the applicable policy. The local **AuthorityContext** is an explicit capability supplied by a trusted integrating caller. It can check aliases, visible inputs, named roles and allowed policies within that boundary. A self-authored identity file is not independent authentication, and different agent names do not establish independent institutions or uncorrelated reasoning.

A6: do not promote validation output. A consistent candidate graph may legitimately report that authority was not checked. Such output is not a release token. Formal scientific decisions require the adopted qualification and separation policies, actual execution evidence where applicable, and appropriate domain review. These institutional and scientific requirements are not supplied by the schema package.

A precise formal expression can align with an intended mathematical target before its proof is complete. Conversely, kernel acceptance cannot rescue a formal statement that expresses a different target. Formal-alignment evidence must therefore name the actual comparison endpoints, rather than borrowing a source-to-argument review that never compared the formal representation.

4 Record families across the lifecycle

The 64 V3 record families cover a planned lifecycle rather than 64 running services. The groups below explain why each family exists and where it hands information to another stage. Appendix A supplies the generated payload and shared-field reference. The groups are an explanatory organization; exact discriminators and references remain those of the schemas.

4.1 Policy and evidence planning: three families

ProcessingProfile, AuthorityPolicy, EvidenceProtocol. A processing profile specifies required stages, axes, success obligations and permitted unresolved outcomes. Authority policy records governance state, role separation, identity requirements and human-review triggers. Evidence protocols state a comparison plan before the result being assessed is selected. Their shared purpose is to make the acceptance question explicit in advance; an authored policy record does not adopt itself.

4.2 Acquisition and source representation: six families

SourceRequest, SourceAcquisition, SourceSnapshot, SourceSpan, SourceTransformation, PaperStructure. The request expresses intent, acquisition records its actual outcome, and the snapshot binds acquired material. Spans locate specific bytes, transformations describe changed representations, and paper structure records document relationships. This separation allows an acquisition failure or uncertain rendering status to remain visible without destroying the request or inventing source content.

4.3 Inventory, scope and obligations: five families

AgentizationPlan, InventoryDiscovery, ScopeFreezeDecision, FrozenInventoryScope, ObligationDisposition. The plan proposes work; discovery records what the source inventory actually contains; an independent scope decision may freeze that work under a profile. Each frozen obligation has a current disposition while historical attempts remain available. Scope is a versioned scientific boundary, not an adjustable progress-bar denominator.

4.4 Scientific and mathematical meaning: seven families

ScientificClaim, DerivedClaim, ClaimComponentMap, SemanticContext, MathematicalDefinition, MathClaimIR, AssumptionContext. These records separate attributed source propositions from system-derived propositions, retain the component correspondence, and describe mathematical and scientific context. MathClaimIR means mathematical claim intermediate representation. A definition can be reused only under a compatible meaning and version. Assumption contexts are explicit so that a conditional derivation can be displayed without silently turning into an unconditional claim.

4.5 Arguments and work history: eleven families

ArgumentNode, InferenceStep, ProofPlan, DependencyBinding, Challenge, ChallengeDisposition, ArgumentSnapshot, ArgumentReview, UpstreamImport, WorkAttempt, WorkCompletion. Nodes and joint inference steps represent an argument; proof plans distinguish routes; bindings identify exact dependencies. Challenges and their dispositions preserve objections and responses. A snapshot supplies a recoverable argument boundary. Imports retain upstream attribution, while attempts and completions retain reported or observed work evidence. Establishing actual execution requires suitable independently supplied receipts. A reply does not automatically resolve a challenge, and a worker completion does not itself establish a theorem.

4.6 Formalization and assessment: twelve families

FormalEnvironment, FormalizationPacket, FormalizationAttempt, FormalCheckRecord, BacktranslationRecord, AlignmentAssessment, EmpiricalEvidence, ReproductionRecord, AxisAssessment, StatusView, CounterexampleRecord, EnvironmentCompatibility. A formalization packet binds a target and context. An attempt records attributed production; a check binds reported or observed evidence to declared targets and an environment. Independent execution receipts are needed to establish which checks actually ran. Backtranslation and alignment inspect meaning at exact endpoints. Empirical and computational evidence remain separate. Counterexamples specify what they contradict; environment checks support a proposed use. A status view aggregates these records without changing their targets or authority.

4.7 Knowledge comparison and reuse: five families

BaselineSnapshot, RelationAssessment, ContributionDelta, ReuseDecision, FrontierItem. A contribution is relative to an explicit baseline. Relations such as equivalence, restriction, extension or refutation require the appropriate endpoint comparisons and evidence. A new proof of a known theorem can be a method contribution without becoming a new theorem; a textually different formulation may add no supported result. Reuse decisions are use-specific and preserve assumptions, environment and conflicts. Frontier items retain the work required before a future decision becomes possible.

4.8 Package, archive and admission: eight families

PaperAgentManifest, ReleaseAudit, ReleaseCertificate, PaperAgentRelease, ArchiveReceipt, AdmissionDecision, KnowledgeSnapshot, KnowledgeIngestionReceipt. The manifest fixes candidate contents; audit and mechanical certification bind that exact manifest; a paper release fixes the resulting package. Archiving preserves the release. A later admission decision selects eligible records for a knowledge snapshot, and an ingestion receipt records the actual transaction. Software merge, package release, archive and knowledge admission are separate events. A successful database write is not an independent scientific review.

4.9 History, migration and observation: seven families

EventRecord, EventCheckpoint, ImpactAnalysis, RevisionRecord, LegacyBinding, QueryReceipt, EvaluationReport. Events and checkpoints retain operational history. Impact analysis identifies potentially affected records along declared dependencies without rewriting old assessments. Revision records preserve changes; legacy bindings retain the source contract of older material. Query receipts bind what was returned under a snapshot. Evaluation reports record measured scope, baselines and limitations rather than converting absent measurements into favourable results.

4.10 Local implementation boundary

The current Python modules implement offline schema and supplied-record validation, local source inspection, frozen-obligation checks, a pinned upstream capture parser, potential dependency analysis and bounded SQLite candidate storage. The local store validates an input set before committing, preserves immutable

object identities and can update a provisional head with compare-and-swap. That operation changes the head only if it still equals the expected previous value, preventing a stale writer from silently replacing a concurrent update.

Historical queries resolve exact records and retain known relevant conflict endpoints. Their responses remain provisional. The implementation trusts its local filesystem and integrating process; it is not a production identity provider, externally witnessed archive or formal admission service. The richer release and scientific lifecycle families remain necessary interfaces even where only their shape and some cross-record rules are currently implemented.

5 Worked example: robustness of magic

5.1 What is being reconstructed

Howard and Campbell define robustness of magic through signed decompositions into pure stabilizer states and state its monotonicity under trace-preserving stabilizer channels [1]. The local source-grounding note identifies the definition, the R3 statement, its proof steps and a separate postselection extension. This section explains the recorded argument and its assumptions; it is not a new independent scientific certification.

For a finite multiqubit system, let $\mathcal{S}_n = \{\sigma_i\}$ denote the pure stabilizer projectors used as decomposition atoms. A density operator ρ has a real signed decomposition

$$\rho = \sum_i x_i \sigma_i, \quad \sum_i x_i = 1, \quad x_i \in \mathbb{R}. \quad (3)$$

The coefficients need not be probabilities: some can be negative. The source defines

$$\mathcal{R}(\rho) = \min_{\{x_i: \rho = \sum_i x_i \sigma_i\}} \sum_i |x_i|. \quad (4)$$

The total absolute coefficient weight measures how much signed weight is needed to represent the state in these free atoms. An ordinary convex mixture has total weight one. The technical obligation to establish feasible decompositions and attainment of the minimum is separate from writing down this definition.

5.2 Why trace preservation gives the contraction

For the selected R3 argument, assume linearity and that the channel maps every input stabilizer atom to a nonnegative, normalized mixture of output stabilizer atoms:

$$\mathcal{E}(\sigma_i) = \sum_j p_{ij} \sigma_j, \quad p_{ij} \geq 0, \quad \sum_j p_{ij} = 1. \quad (5)$$

This is mixture preservation, not the stronger claim that every input atom maps to a single atom. It is therefore important to keep the two indices and normalization condition in the recorded mathematical target.

Take an optimal input decomposition x . Linearity and regrouping give the feasible output decomposition

$$\mathcal{E}(\rho) = \sum_j x'_j \sigma_j, \quad x'_j = \sum_i x_i p_{ij}. \quad (6)$$

The definition, triangle inequality, nonnegativity and row normalization yield

$$\begin{aligned} \mathcal{R}(\mathcal{E}(\rho)) &\leq \sum_j \left| \sum_i x_i p_{ij} \right| \\ &\leq \sum_{i,j} |x_i| p_{ij} \\ &= \sum_i |x_i| \underbrace{\sum_j p_{ij}}_1 \\ &= \mathcal{R}(\rho). \end{aligned} \quad (7)$$

The final equality uses optimality of the input decomposition. For an arbitrary feasible x , the preceding argument gives only the upper bound $\sum_i |x_i|$, which may exceed $\mathcal{R}(\rho)$. An alternative proof could use an infimum and a limiting argument, but that is a distinct reconstruction with its own obligations.

The physical picture is simple: a free operation redistributes each signed input weight among free output atoms using nonnegative proportions that sum to one. Redistribution cannot increase the total absolute weight, and cancellations can reduce it. The assumptions in Eq. (5) are precisely what makes this picture match the algebra.

5.3 Why postselection is a separate claim

A selected measurement branch is represented before normalization by a trace-nonincreasing map $\mathcal{E}_b$. Its success probability is

$$q_b = \text{Tr } \mathcal{E}_b(\rho), \quad \rho_b = \mathcal{E}_b(\rho)/q_b \quad (q_b > 0). \quad (8)$$

Normalization divides by a state-dependent probability. The row sums used in Eq. (7) are no longer all one for an individual branch. The trace-preserving proof therefore does not directly show $\mathcal{R}(\rho_b) \leq \mathcal{R}(\rho)$ for each normalized selected outcome.

The source separately discusses an average/postselection extension and points to another argument. A probability-weighted quantity, such as an average over outcomes, asks a different question from the robustness of a single conditional branch. AgtXiv keeps this assertion on a separate route with its own source span and open obligations; it does not count it as discharged by R3.

5.4 Exact anchors and observed evidence

Source segment	TeX lines	Original byte interval
Definition	75–79	[9218, 10126)
R3 statement	225	[33249, 33406)
Channel action	251–255	[35064, 35619)
Output mixture	256–259	[35619, 35847)
Norm contraction	260–265	[35847, 36153)
Separate extension	266	[36153, 36418)

The original TeX is `Robustness_main_appendix.tex`, SHA-256 `48adf581a5f8353bb1125c75b8e0a075ff7ba79917f2a12d9f7e0f4ad1a8312d`. These anchors were read from the retained source and source-review note. The historical candidate capture contains 14 files totalling 1,542,636 bytes. Its 548 main-text lines were assigned 240 tentative claim cues, 204 non-claim cues and 104 unknown-text items. These numbers describe lexical coverage, not scientific claim counts.

The recorded output contains 35 candidate records, one unadopted context policy, six proposed interpretations, five dependency bindings, twelve frontier items and nine open/deferred candidate obligations. A fresh database import, a separate-process read-only reopen and byte-identical replay under the same known observation were recorded. No formal frozen scope, scientific assessment or knowledge admission was produced. Supplementary numerical matrices and solver evidence referenced by the paper were not found in the inspected source package; this observation does not establish their absence from every remote location.

Evidence boundary. The candidate chain can expose the optimal-decomposition assumption and keep post-selection separate. This is evidence of useful representation and traceability. The mathematical obligations, source-to-formal alignment, full-paper semantic coverage and formal scientific admission remain open in this V3 candidate record.

6 New-paper web and API architecture

6.1 Purpose and implementation status

The release interface must accept a new arXiv paper and perform a real bounded analysis. This operational requirement is distinct from the precomputed V1 demonstration and from independent scientific assessment.

The primary user path is submission, acquisition, processing, visible progress or failure, and a result linked to source evidence. A successful run may still report incomplete extraction or no supported scientific conclusions.

Evidence boundary. The routes and state behaviour below are bound to the local implementation and an actual HTTP submission of HepLean, arXiv:2405.08863v1. The retained verification covers task/result schemas, exact-version acquisition, repeated result reads and six unassessed axes. Local browser checks additionally cover real submissions, refresh recovery, narrow-screen layout and keyboard interaction. Public deployment remains a separate release check.

The selected implementation uses a plain HTML, CSS and JavaScript front end under `web/`, with an ECMAScript-module Cloudflare Worker as the server. D1 stores task metadata and stage transitions; R2 retains acquired source bytes and generated results. An independent intake module performs arXiv input handling, real acquisition, bounded archive and TeX parsing, candidate source-span extraction and local-reference dependency extraction. These are source-processing operations; no language-model scientific approval is supplied by this path.

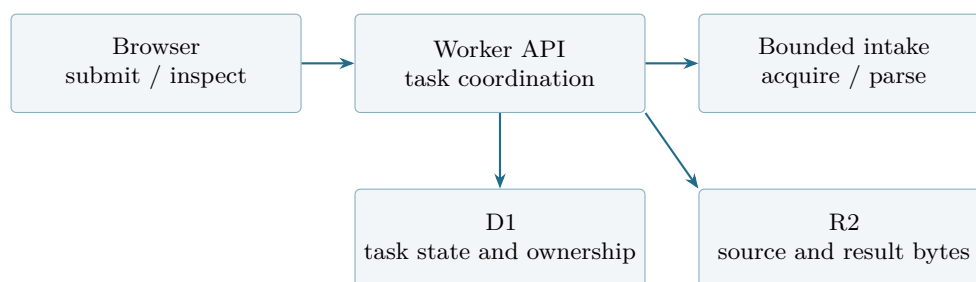


Figure 2: Operational architecture for new-paper processing. Task metadata and byte objects serve different roles. Cross-service failures require explicit recovery; this diagram does not imply a single atomic transaction across D1 and R2.

6.2 Required transport contract

The API has a major-version path prefix `/api/v1` and currently returns `api_version:"1.0"`. The inspected routes are:

Operation	Request and response meaning
GET <code>/api/v1/health</code>	Service configuration state: READY if both stores are bound, otherwise READ_ONLY ; no scientific assessment.
GET <code>/api/v1/library?q=...</code>	Curated paper records, optionally filtered by a case-insensitive query of at most 200 characters. This endpoint returns the selected finite library, not a paginated global arXiv index.
GET <code>/api/v1/papers/SLUG</code>	One curated paper, or 404. A missing curated paper can be submitted through the jobs endpoint.
GET <code>/api/v1/schemas</code>	The exposed schema catalog and its bundle identity.
POST <code>/api/v1/jobs</code>	JSON object with exactly one string field, arxiv . Returns 202 with a task view and Location pointing to the task.
GET <code>/api/v1/jobs/JOB_ID</code>	Persisted task view: state, timestamps, revision, parent, result location and structured error where applicable.

Operation	Request and response meaning
GET /api/v1/jobs/JOB_ID/result	Completed result bytes; 409 before completion; 503 for a missing or hash-mismatched retained result.
POST /api/v1/jobs/JOB_ID/retry	Empty JSON object. Only failed or interrupted tasks are retryable. Creates a new task with <code>parent_id</code> ; it does not overwrite the previous attempt.

The capitalized segments `SLUG` and `JOB_ID` are route variables, not literal path components. JSON requests are limited to 2 KiB and require `application/json`. Cross-origin browser POST requests are rejected. Error responses contain a code, message and request identifier; a successful transport status does not imply a supported scientific claim.

W1: submit intent. The submission request contains exactly the `arxiv` field, accepting an arXiv identifier or recognized arXiv URL; no processing-options field is currently supported. The server validates and normalizes the identifier before acquisition. An arbitrary user-supplied URL is not an authorized fetch destination. The response supplies a stable task identifier and the means to inspect its state. The acquired version and source identity are recorded separately from the requested identifier.

W2: report actual state. A task exposes its current stage, update or revision identity, and structured errors. Progress reflects persisted work transitions rather than a client-side timer. Concurrent attempts must not both take ownership of the same stage; a compare-and-swap transition checks the expected state before changing it. Interrupted or expired attempts require an explicit retry/recovery policy rather than indefinite apparent progress.

The persisted success sequence is `QUEUED` → `FETCHING` → `ANALYZING` → `COMPLETED`, with `FAILED` for a recorded processing failure. A non-terminal task with no update for more than 120 seconds is projected as `INTERRUPTED` when read. This derived view does not rewrite its stored state or confirm that the worker has stopped. The earlier attempt may still complete. A retry records a separate task and parent link, preserving both attempts and their possible results. All task views explicitly report `scientific_assessment:"NOT_PERFORMED"`.

W3: retain evidence. Results bind the resolved source identity, original byte digest, processing version, extraction limits, candidate records and warnings. Candidate relations preserve their extraction basis. A local TeX reference is evidence of an explicit textual reference; it is not automatically a validated mathematical implication. Reading a job result must not mutate its source record or scientific status.

Source archives are retained privately under keys derived from their SHA-256. Result JSON is likewise stored under its own byte digest, which is checked on result retrieval. A source digest and a result digest identify different byte objects. The result includes requested and resolved source information, retrieval time, archive digest, engine version, candidate spans and explicit remaining limitations. In the HTTP result, `transport.archive_sha256` uses the `sha256:` prefix required by the response schema. Candidate assessment entries remain `NO_ASSESSMENT`; they are not `V3 AxisAssessment` records.

W4: expose failure. Invalid input, unknown work, source unavailability, unsupported source format, archive rejection, resource limits and processing failure need separate error codes or documented details. An error response must not expose internal credentials or arbitrary filesystem paths. The client displays which stage failed and whether retry is meaningful. A failed parse must not leave a successful-result label.

W5: document the actual interface. The OpenAPI 3.1 description is published as a build artifact at `web/public/api/v1/openapi.json`, exposed at `/api/v1/openapi.json`. It defines routes, methods, request/response schemas, status codes and polling/retry semantics. The strict intake result schema is maintained at `src/agtxiv_web/analysis.schema.json`. Task retention and production access policy still require operational decisions. The task-result contract remains distinct from V3 scientific records; similar field names do not make an operational JSON object a sealed V3 record.

6.3 Bounded processing and separation from scientific work

Input acquisition must constrain network destinations, redirects, response size and time. Archive processing must constrain expanded bytes, entry count, path traversal, links and malformed members. TeX inspection is

static and must not execute source-controlled commands. The configured limits belong in API documentation and actual server-side checks, not only in front-end validation.

The server uses a 20-second configured timeout for both acquisition and analysis, and a conditional update to a shared submission clock permitting a new task no more often than every 3.5 seconds. A conditional SQL insertion admits a task only while fewer than three non-terminal tasks have recent updates. The 120-second cutoff bounds recently active records, not every worker that may still run after a stale projection. Intake bounds downloads to 8 MiB, expansion to 16 MiB, one file to 2 MiB, total TeX text to 4 MiB, tar entries to 1,024, files to 512 and candidates to 240. Other limits bound labels, references, includes, structural spans and hashing work. Each result records the configured limits under its defined response schema. Earlier browser receipts retain their original 30-second engine field; the final HTTP receipt records the current 20-second configuration.

Source-span candidates provide a route from a displayed item to the bytes examined. They do not establish complete semantic claim discovery, source-faithful paraphrase or theorem validity. If a run cannot resolve macros, parse a document class or interpret a figure, that uncertainty belongs in the result. Restricting work is acceptable when the limit is explicit; reporting that limited work as complete scientific analysis is not.

The web service therefore has its own operational lifecycle, while V3 describes a richer scientific lifecycle. D1 and R2 here serve the bounded web task implementation. Their use does not silently adopt or replace the V2 production authority proposal, which distinguishes reviewed Git material, canonical replay bundles, operational transactions and disposable projections. A future adapter must bind a task's actual artifacts, producer and evidence to the relevant V3 records before any stronger scientific interpretation is made.

6.4 Reader and interface acceptance

A reader should be able to submit a new paper, inspect a persisted task after refreshing the page, open the result, locate a source passage and identify missing evidence. The graph must distinguish workflow steps from paper-specific dependencies. Keyboard navigation, visible focus, narrow-screen layout, readable formulas and text labels for status colours are part of the acceptance path.

The signed-weight interaction accompanying R3 is an algebraic illustration of nonnegative redistribution and cancellation. Its displayed coefficients and transition matrix are not a specified quantum state and channel, and its total absolute weight is not a computed state robustness. The interface must keep this distinction visible alongside the optimal-decomposition and trace-preserving assumptions.

Final integration checks must include a new paper outside the precomputed example list, actual source acquisition, result evidence, a meaningful failure case, persisted task lookup and an independent comparison of API and displayed states. Successful local checks are not evidence of public deployment; the release record must name the actual running location or provide an explicitly local run procedure.

The local adapter in `web/dev.mjs` uses Node's SQLite support and a private local object directory to exercise the same handler. Its default address is `http://127.0.0.1:8787`. It is a development environment, not a measurement of the production Cloudflare deployment. The server entry point can be started from the repository root with `nodeweb/dev.mjs` after the required runtime and web assets are available.

6.5 Retained live HTTP evidence

The local run recorded in `docs/evidence/publishable-release/heplean-http-final/verification.json` submitted 2405.08863v1 to the running HTTP service on 7 September 2026 at 23:03 UTC. It observed QUEUED, then COMPLETED with revision 3, and retrieved identical result bytes twice. Polling did not capture each intermediate stage separately. The exact version 1 archive contains three source files, including two TeX files; the extractor returned 15 candidates. All six axes remained NO_ASSESSMENT, and scientific_acceptance remained false.

The receipt reports successful job-schema, full result-schema, exact-version, retained-source, repeated-read and six-axis checks. The result in `heplean-http-final/result.json` has a nonempty entity tag matching its byte digest and reports `limits.timeoutMs=20000`. Earlier receipts remain historical evidence. These checks establish a real processing path, not extraction accuracy or scientific endorsement of HepLean.

Browser checks initially exposed an incorrect MIME type for the root HTML page, observed as ERR_BLOCKED_BY_CLIENT. The root response was corrected to `text/html` and a regression test added. The local

browser then rendered the white interface and completed a form submission of HepLean with 15 candidates and three source files. The browser’s two exposed WebMCP tools were discovered; reading the submitted task agreed with the API, a new tool submission of 1706.03762 resolved to version 7 and completed with 11 candidates and 23 inventoried source files, and an invalid external URL was rejected by both tool and form paths.

The receipt `docs/evidence/publishable-release/browser-verification.json` records these checks and additional refresh recovery, keyboard interaction with the signed-weight slider, real schema-field expansion and absence of captured browser errors. A 390-by-844 viewport override produced equal measured client and document-scroll widths of 375 pixels, with no page-wide horizontal overflow in the inspected flow. This is scoped local interface evidence, not a complete accessibility audit. The retained task/result JSON files accompany the receipt under `browser/`. Report-download verification and an actual public deployment URL remain separate next steps.

A separate local handler check submitted 20 concurrent same-origin requests: one received 202, nineteen received 429, and one processing task was scheduled. Its receipt is `docs/evidence/publishable-release/intake-checks/concurrency-local.json`. This check injected a synthetic TeX upstream and used the local SQLite adapter; it tests the observed admission behaviour, not Cloudflare production concurrency or real upstream throughput. The accompanying retained logs report 14 engine tests, 23 response-schema tests and six handler tests passing. These scoped results do not replace a full repository or production release check.

7 Implementation evidence, maintenance and release

7.1 What has been checked

The historical `docs/evidence/v3-real-candidate/verification.json` records 710 V3 tests with no failures, errors or skips on 7 September 2026. Its scope also includes 70 generated package files, the pinned upstream protocol, bundle validation, compilation and 106 unchanged historical inputs. These figures do not certify the current release.

Rejection tests cover malformed or stale identities, escaped assumptions, conditional promotion, unsupported reuse, misleading evidence wrappers, historical forks and transaction recovery. They constrain incorrect bindings in supplied records; they do not measure semantic extraction accuracy or establish independent scientific review.

Current HTTP evidence is scoped in Section 6. The aggregate gate encountered a pre-existing dirty frozen-baseline mismatch against `HEAD`, rather than a newly failing test. The independent full test run was outstanding at this checkpoint. Historical tests and local HTTP success do not complete that gate. Report compilation and packaging have a separate receipt at `docs/evidence/publishable-release/report-validation.json`.

Evidence	Appropriate interpretation
Schema/record validation	The implemented constraints hold for the supplied objects and exact bundle.
Parser/source tests	Bounded input handling and selected source-location behaviours match the tested cases.
Candidate replay	The recorded local observation can be re-read or replayed with the stated byte identity.
Historical Lean evidence	Only its named declaration, environment, premise and alignment scope; not all V3 scientific obligations.
Browser/API test	The tested user operation reaches the recorded service result; not scientific approval.

7.2 Compatibility and document authority

M1: preserve old contracts. V1 and V2 records retain their original schema and evidence scope. The root pilot release manifest is historically `accepted_release=false`; its scientific release gate is blocked. That state is not changed by introducing V3 schemas or a new website. A legacy adapter can expose older

records with explicit version labels, but cannot turn three old assessment dimensions into six newly assessed dimensions.

M2: make authority visible. The V3 proposal, V1/V2 specifications, pending Charter, research studies and implementation receipts serve distinct roles. This report describes implementation and maintenance; it does not ratify the Charter. Its appendix derives fields from the bound schemas rather than redefining them.

The existing static Pages builder previously selected the old mathematics-pipeline demonstration, while newer visual prototypes and V3 candidates were outside that path. Publication must verify the actual assembled entry point, not merely the existence of a new page in the repository. The new service also requires a running processing backend; a static website alone cannot fulfil the new-paper analysis requirement.

7.3 Recoverable repository organization

M3: preserve before cleaning. A cleanup first records original paths, destinations, sizes, hashes and reasons. Unique historical byte stores require preservation before deleting temporary files. Fixed historical evidence is not rewritten to conceal changed paths or inputs. A new relocation record explains where unchanged bytes can be found.

The first organization pass copied five complete candidate-history files from the temporary capture directory to an ignored `local-archive/` directory and kept the originals. The destination bytes matched the historical closure receipt; both SQLite copies passed a read-only structural quick check. Four historical presentation exports were moved from the repository root into `docs/archive/presentations/2026-08/`. The nine archived files total 16,488,328 bytes. The archive manifest records exact hashes and reversible path mappings.

A second pass moved 17 files from the two superseded interface prototypes into `docs/archive/prototypes/`. Three README files gained explicit relocation notes; other file bytes were unchanged, and the original README content remains recoverable as verified prefixes. The combined manifest has 26 entries and 18,943,027 destination bytes. Local prototype resources, anchors and graph-data references were checked after relocation.

The operation left scientific source caches, fixed reference inputs, active slides, schema packages and Lean caches untouched. The local copy reduces the risk of losing historical evidence during temporary-directory cleanup; it is not an off-device backup or independent tamper-proof archive. Archiving material locally does not grant permission to redistribute it in a public website or release.

7.4 Build and release procedure

R1: bind the candidate. Record the exact source revision, dependency locks, generated-input identities, migration notes and output digests. Separate the software version from record contracts, API version and paper releases. A clean reproducible build is a software property, not scientific acceptance.

R2: run appropriate checks. The release check set should include the repository aggregate validation, schema and upstream-generation drift checks, V3 contract checks, the new intake's resource and failure tests, API/task integration, browser acceptance and report compilation. A known blocker is reported as such, rather than silently skipped or counted as a scientific pass. Expensive formal builds use their defined environment and scope.

R3: publish the intended bytes. Build the website and service from the reviewed candidate. Verify deep links, downloads, subpath behaviour, error states and the actual deployed endpoint when deployment occurs. Public artifacts must be selected deliberately; local research caches and preserved byte stores are not default release content. Root licence, contributor and production-governance decisions remain governed by the project's actual policies.

R4: preserve operations. Document service configuration, data retention, access boundaries, failure recovery and backup/restore procedures. Recovery must account for both task metadata and byte objects. A task that points to a missing result needs an explicit repair path, and an unreferenced byte object needs a retention policy. Production durability and throughput require separate measurements.

R5: retain an honest handoff. The authorized work window ends at 03:00 Europe/Amsterdam on 8 September 2026, equivalent to 01:00 UTC. Work remaining at that deadline is saved with concrete files, evidence, failure reasons and next operations. The deadline does not make an unfinished implementation or

scientific task complete.

7.5 Reproducible report commands

The report appendix generator reads the source schemas and writes only its own generated files. It exposes a no-write comparison mode. From the repository root:

```
.venv/bin/python docs/technical-report/generate_appendix.py
.venv/bin/python docs/technical-report/generate_appendix.py --check
bash docs/technical-report/build.sh
```

The PDF is written to `docs/technical-report/build/main.pdf`. Build intermediates are ignored. A portable source archive includes the TeX, BibTeX, generated appendix and build script. Outside the repository, `bashbuild.sh--frozen-appendix` compiles that included snapshot without claiming to check drift against absent schema inputs. The default repository build retains the generation check. The layout is rendered and reviewed separately from compilation, because successful TeX execution does not guarantee readable tables or diagrams.

8 Boundaries, evaluation and next work

The current result makes scientific work traceable while checking selected misbindings. The robustness example explains the need: trace preservation, optimal decomposition and postselection each change the argument and must remain inspectable.

Lexical cues do not establish complete semantic coverage, byte identity does not establish faithful interpretation, review fields do not authenticate independence, and local transactions do not establish production durability. Intake supplies candidates while these scientific and operational questions remain open.

Future evaluations need fixed papers, scope and evidence standards. Useful measurements include omitted conditions, correct source locations, rejected misuse, resolution time and recovery. Efficiency claims require a measured baseline. The original 58-task roadmap remains in force; the seven-workstream release plan does not discharge unperformed scientific work.

8.1 Claim–evidence map

Claim	Source of evidence	Status in this report
64 record families and 19 shared structures	Bound catalog, shared schema and generated appendix inventory	Directly enumerated
Six questions are represented separately	Axis schema, projection code and invariant ledger	Contract/local mechanism
R3 assumptions and norm contraction	Original TeX and exact source-review anchors	Source-grounded reconstruction
Historical candidate rereading	Closure receipt and retained database bytes	Historical local check
710 V3 tests passed at the stated checkpoint	Stored verification JSON and JUnit	Historical test result
New-paper source processing	HepLean HTTP receipt and local browser verification	Local paths checked; deployment pending
Scientific acceptance and productivity gain	No corresponding completed evaluation	Not established

Reference

- [1] Mark Howard and Earl T. Campbell. Application of a resource theory for magic states to fault-tolerant quantum computing. arXiv:1609.07488v2 [quant-ph], 2017. URL <https://arxiv.org/abs/1609.07488v2>. Version 2, 15 March 2017; retained source TeX inspected.

A Generated contract field reference

This appendix is generated from the exact experimental JSON Schema package. It documents all registered payload fields and shared structures. The envelope is shown once. Required markers are local to the containing object; conditional branches, exact regular expressions and extended enumerations remain defined by the original schemas. A field being present does not establish that its producer or scientific reviewer has been implemented.

The source manifest SHA-256 is 9f48cc2c992b28f72a6592a50a0240203bd75b97eee036660a7bc0c80f597854. The report-side input inventory is stored in `generated/schema-inputs.json`.

A.1 Common record envelope

Field	Meaning and local constraints
<code>record_type</code> (required)	The exact discriminator of this record family; its constant is specified by each family schema. string
<code>record_id</code> (required)	Stable identity; semantic changes may require a new lineage. string; maxLength=512; pattern enforced (see schema)
<code>revision</code> (required)	Exact immutable version within the lineage. integer; minimum=1; maximum=9007199254740991
<code>created_at</code> (required)	Attributable creation time in UTC. string; maxLength=40; format=date-time; pattern enforced (see schema)
<code>data_class</code> (required)	Evidence origin, not an authority status. one of RESEARCH, OBSERVED, SYNTHETIC
<code>schema_bundle_hash</code> (required)	Exact manifest hash of the schema bundle. string; minLength=71; maxLength=71; pattern enforced (see schema)
<code>producer</code> (required)	Actual attributed producer and execution boundary. Producer
<code>policy_ref</code> (required)	Exact authority policy binding. allOf: RecordRef; constrained object
<code>input_refs</code> (required)	Explicit exact inputs used to construct this record. array of RecordRef; maxItems=10000
<code>content_hash</code> (required)	Canonical record hash excluding this one field only. string; minLength=71; maxLength=71; pattern enforced (see schema)
<code>payload</code> (required)	Family-specific fields, documented separately for all registered record families below. object; closed object

A.2 Shared value structures

A.2.1 RecordRef

An exact immutable record reference, never a latest-version selector.

Field	Meaning and local constraints
record_type (required)	Exact registered record discriminator. enum (64 exact values; see schema); minLength=1; maxLength=4096
record_id (required)	Stable record lineage identity. string; maxLength=512; pattern enforced (see schema)
revision (required)	Exact immutable revision. integer; minimum=1; maximum=9007199254740991
content_hash (required)	Canonical complete-record hash excluding only content_hash. string; minLength=71; maxLength=71; pattern enforced (see schema)

A.2.2 ArtifactRef

Identity of supplied raw bytes; a locator is not integrity evidence.

Field	Meaning and local constraints
artifact_id (required)	Stable artifact identity. string; maxLength=512; pattern enforced (see schema)
sha256 (required)	SHA-256 of unmodified raw bytes. string; minLength=71; maxLength=71; pattern enforced (see schema)
byte_size (required)	Exact byte length. integer; minimum=0; maximum=9007199254740991
media_type (required)	Declared media type. string; minLength=1; maxLength=4096
path_hint (optional)	Relative display locator, never an authority selector. string; minLength=1; maxLength=4096; pattern enforced (see schema)

A.2.3 Producer

Attributed producer and its visible execution boundary.

Field	Meaning and local constraints
principal_id (required)	Actual principal identity asserted by the execution record. string; maxLength=512; pattern enforced (see schema)
role (required)	Role used for this operation. enum (19 exact values; see schema)
actor_kind (required)	Producer class, not an assurance level. one of HUMAN, AGENT, MECHANICAL_SERVICE
identity_assurance (required)	How identity was established; strings alone are not authentication. one of DECLARED, LOCALLY_ATTESTED, EXTERNALLY_ATTESTED
execution_id (required)	Exact attributed execution. string; maxLength=512; pattern enforced (see schema)
model (optional)	Model identifier when an agent is used. string; minLength=1; maxLength=4096

Field	Meaning and local constraints (continued)
visible_input_refs (required)	Exact records visible to the actor. array of RecordRef; maxItems=10000
identity_evidence (required)	Independently supplied identity evidence. array of ArtifactRef; maxItems=10000

A.2.4 ReviewContext

Declared independence plus exact reviewed input; a runtime must verify it.

Field	Meaning and local constraints
reviewed_refs (required)	Exact records assessed by this decision. array of RecordRef; minItems=1; maxItems=10000
producer_principal_ids (required)	All actual producers of the reviewed content. array of string; minItems=1; maxItems=10000
independence (required)	Review boundary, not a truth guarantee. one of UNESTABLISHED, ROLE_SEPARATED, INDEPENDENTLY_ATTESTED
conflicts (required)	Known conflicts of interest or shared-model limitations. array of string; maxItems=10000
method (required)	Public review procedure. string; minLength=1; maxLength=65536
evidence_refs (required)	Evidence supporting the review. array of RecordRef; maxItems=10000

A.2.5 Condition

A material condition with provenance and interpretation.

Field	Meaning and local constraints
condition_id (required)	Stable condition identity within its parent record. string; maxLength=512; pattern enforced (see schema)
statement (required)	Complete condition including range and quantifiers. string; minLength=1; maxLength=65536
origin (required)	Attribution of the condition. one of SOURCE_EXPLICIT, SOURCE_RECONSTRUCTED, AGENT_ADDED, IMPORTED
source_span_refs (required)	Exact supporting source spans. array of allOf: RecordRef; constrained object; maxItems=10000

A.2.6 SourceUnit

An inventoried source object and its activity in the selected rendering.

Field	Meaning and local constraints
unit_id (required)	Source inventory item identity. string; maxLength=512; pattern enforced (see schema)
relative_path (required)	Canonical relative path. string; minLength=1; maxLength=4096; pattern enforced (see schema)
artifact (required)	Exact unmodified source bytes. ArtifactRef
activity (required)	Participation in the designated rendering. one of ACTIVE, DORMANT, UNKNOWN
kind (required)	Source object role. one of TEX, PDF, BIBLIOGRAPHY, FIGURE, TABLE, CODE, DATA, OTHER

A.2.7 StructureEdge

Document structure only, not mathematical inference.

Field	Meaning and local constraints
source_unit_id (required)	Including or containing source unit. string; maxLength=512; pattern enforced (see schema)
target_unit_id (required)	Included or contained source unit. string; maxLength=512; pattern enforced (see schema)
relation (required)	Document relation. one of INCLUDES, CONTAINS, REFERENCES, CAPTION_OF
activity (required)	Static activity observation. one of ACTIVE, DORMANT, UNKNOWN
locator (required)	Location of the relation in the source. string; minLength=1; maxLength=4096

A.2.8 Obligation

One immutable processing obligation in a frozen denominator.

Field	Meaning and local constraints
obligation_id (required)	Identity preserved across attempts. string; maxLength=512; pattern enforced (see schema)
target_refs (required)	Exact known targets; empty only for unresolved discovery items. array of RecordRef; maxItems=10000
source_unit_ids (required)	Source units covered by the obligation. array of string; minItems=1; maxItems=10000
stage (required)	Required work stage. enum (11 exact values; see schema)
requirement (required)	Accounting versus a required successful result. one of ACCOUNT_FOR, SUCCESS_REQUIRED

Field	Meaning and local constraints (continued)
applicable_axes (required)	Axes to which this work is relevant. array of one of source_fidelity, mathematical_correctness, formal_alignment, semantic_applicability, empirical_support, computational_reproducibility; maxItems=10000
reason (required)	Why this obligation belongs in scope. string; minLength=1; maxLength=65536

A.2.9 Component

An independently attributable part of a source proposition.

Field	Meaning and local constraints
component_id (required)	Component identity within the claim. string; maxLength=512; pattern enforced (see schema)
text (required)	Complete component meaning. string; minLength=1; maxLength=65536
role (required)	Semantic role. one of CONCLUSION, ASSUMPTION, DEFINITION, MODEL, APPROXIMATION, EVIDENCE, ATTRIBUTION, LIMITATION
source_span_refs (required)	Exact source anchors. array of allOf: RecordRef; constrained object; minItems=1; maxItems=10000

A.2.10 ComponentMapping

Total mathematical, semantic and unresolved accounting for one component.

Field	Meaning and local constraints
component_id (required)	Exact component in the source claim. string; maxLength=512; pattern enforced (see schema)
math_refs (required)	Mathematical representations. array of allOf: RecordRef; constrained object; maxItems=10000
semantic_refs (required)	Scientific meanings retained separately. array of allOf: RecordRef; constrained object; maxItems=10000
evidence_refs (required)	Evidence associated with this component. array of RecordRef; maxItems=10000
disposition (required)	Component disposition. one of MAPPED, RESIDUAL, UNRESOLVED, NON_CLAIM
residual (required)	Exact meaning still outside the mapped representations. string; maxLength=65536

A.2.11 MathObject

A mathematical carrier with definitions, units and conventions.

Field	Meaning and local constraints
symbol (required)	Fully scoped symbol. string; minLength=1; maxLength=4096
object_type (required)	Mathematical carrier or type. string; minLength=1; maxLength=65536
definition_refs (required)	Exact imported definitions. array of allOf: RecordRef; constrained object; maxItems=10000
unit (required)	Unit, or empty for a dimensionless object. string; maxLength=65536
convention (required)	Normalization and interpretation convention. string; maxLength=65536

A.2.12 Quantifier

Ordered binder; order is semantically significant.

Field	Meaning and local constraints
kind (required)	Quantifier kind. one of FORALL, EXISTS, UNIQUE_EXISTS
variable (required)	Bound variable name. string; minLength=1; maxLength=4096
domain (required)	Quantified domain with restrictions. string; minLength=1; maxLength=65536

A.2.13 FormalDeclaration

An exact declaration observed in one formal environment.

Field	Meaning and local constraints
name (required)	Fully qualified declaration name. string; minLength=1; maxLength=4096
statement (required)	Elaborated or inspected declaration type. string; minLength=1; maxLength=65536
source_artifact (required)	Exact declaration source bytes. ArtifactRef
is_axiom (required)	Whether the declaration itself is an axiom. boolean
axioms (required)	Transitive axiom names reported by the checker. array of string; maxItems=10000
dependencies (required)	Exact declaration dependency names in the fixed environment. array of string; maxItems=10000

A.2.14 ParameterComparison

Explicit comparison of a reusable result with its target use.

Field	Meaning and local constraints
dimension (required)	Compared dimension. one of OBJECT, DEFINITION, QUANTIFIER, ASSUMPTION, CONCLUSION, UNIT, CONVENTION, REGIME, EVIDENCE, ENVIRONMENT
source_value (required)	Meaning on the reusable side. string; minLength=1; maxLength=65536
target_value (required)	Meaning required by the target. string; minLength=1; maxLength=65536
relation (required)	Assessed comparison, not text similarity. one of EQUIVALENT, SOURCE_STRONGER, TARGET_STRONGER, INCOMPARABLE, UNKNOWN
witness_refs (required)	Evidence for the comparison. array of RecordRef; maxItems=10000

A.2.15 AdmissionItem

One exact candidate and its admission decision.

Field	Meaning and local constraints
candidate_ref (required)	Exact proposed knowledge record. RecordRef
decision (required)	Admission disposition, not truth. one of ACCEPT, REJECT, BLOCK
assessment_refs (required)	Relevant exact assessments. array of allOf: RecordRef; constrained object; maxItems=10000
reason (required)	Public admission rationale and conditions. string; minLength=1; maxLength=65536

A.2.16 ImpactTarget

One affected target with a typed and axis-limited reason.

Field	Meaning and local constraints
target_ref (required)	Exact affected target. RecordRef
axes (required)	Axes needing review. array of one of source_fidelity, mathematical_correctness, formal_alignment, semantic_applicability, empirical_support, computational_reproducibility; minItems=1; maxItems=10000
proof_plan_ref (required)	Affected proof route, if applicable. anyOf: allOf: RecordRef; constrained object; null
reason (required)	Why this target needs re-evaluation. string; minLength=1; maxLength=65536

A.2.17 QueryAssertion

Public answer text backed by exact records.

Field	Meaning and local constraints
text (required)	One load-bearing assertion in the answer. string; minLength=1; maxLength=65536
support_refs (required)	Exact records supporting the wording. array of RecordRef; minItems=1; maxItems=10000
limitations (required)	Scope and unresolved qualifications. array of string; maxItems=10000

A.2.18 Metric

A measured value with a frozen denominator and comparison scope.

Field	Meaning and local constraints
name (required)	Metric name, never a paper truth or quality score. string; minLength=1; maxLength=4096
measurement_status (required)	Whether a value was observed and a ratio can be defined. one of NOT_MEASURED, MEASURED, UNDEFINED
numerator (required)	Observed count; null when not measured. anyOf: integer; null
denominator (required)	Frozen eligible count; null when unknown and zero only for a measured empty set. anyOf: integer; null
unit (required)	Measurement unit. string; minLength=1; maxLength=4096
method (required)	Measurement protocol and scope. string; minLength=1; maxLength=65536

A.2.19 PdfRegion

A page-local locator in an exact PDF, not a fabricated raw-byte text span.

Field	Meaning and local constraints
page_index (required)	Zero-based page index. integer; minimum=0; maximum=9007199254740991
x0 (required)	Exact decimal left coordinate in normalized page units. string; minLength=1; maxLength=4096
y0 (required)	Exact decimal top coordinate. string; minLength=1; maxLength=4096
x1 (required)	Exact decimal right coordinate. string; minLength=1; maxLength=4096
y1 (required)	Exact decimal bottom coordinate. string; minLength=1; maxLength=4096

A.3 Record families and payloads

A.3.1 ProcessingProfile

Versioned work obligations, including required success and permitted unresolved work.

Schema: processing-profile.schema.json. **Roadmap:** P0.2, P1.3.

Field	Meaning and local constraints
name (required)	Named V3 processing profile. one of V3_SOURCE_MAP, V3_ARGUMENT_AUDIT, V3_FORMAL_SUPPORT
required_stages (required)	Stages every applicable scope must account for. array of string; minItems=1; maxItems=10000
success_required_stages (required)	Stages whose required targets must actually succeed. array of string; maxItems=10000
required_axes (required)	Applicable axes whose assessments are required. array of one of source_fidelity, mathematical_correctness, formal_alignment, semantic_applicability, empirical_support, computational_reproducibility; minItems=6; maxItems=6; uniqueItems=True
allowed_terminal_outcomes (required)	Explicit permitted unresolved execution outcomes. array of string; minItems=1; maxItems=10000
empirical_requirement (required)	Empirical work obligation. one of ACCOUNT_FOR, REVIEW_REQUIRED
computation_requirement (required)	Computational work obligation. one of ACCOUNT_FOR, REPRODUCTION_REQUIRED
non_implications (required)	What completion of this profile does not establish. array of string; minItems=1; maxItems=10000

A.3.2 AuthorityPolicy

Versioned authority, separation and review requirements.

Schema: authority-policy.schema.json. **Roadmap:** P0.1, P0.3, P1.6.

Field	Meaning and local constraints
charter_identity (required)	Charter identity used as the design basis. constant AgtXIV-Charter/1.0
charter_state (required)	Actual ratification state under governance. one of PROPOSED, ADOPTED
ratification_commit (required)	Qualifying adoption commit only after ratification. anyOf: string; null
minimum_identity_assurance (required)	Minimum runtime identity evidence for authoritative decisions. one of LOCALLY_ATTESTED, EXTERNALLY_ATTESTED
separated_role_pairs (required)	Explicit incompatible roles on the same assessed artifact. array of string; minItems=1; maxItems=10000
human_review_triggers (required)	Scientific or governance conditions requiring a human reviewer. array of string; minItems=1; maxItems=10000
allowed_principal_ids (required)	Principals authorized in this exact policy scope. array of string; maxItems=10000

A.3.3 SourceSnapshot

Immutable source bytes and acquisition boundary for one paper version.

Schema: `source-snapshot.schema.json`. **Roadmap:** P2.1.

Field	Meaning and local constraints
<code>paper_id</code> (required)	Exact bibliographic work identifier. string; minLength=1; maxLength=4096
<code>paper_version</code> (required)	Explicit source version, never latest. string; minLength=1; maxLength=4096
<code>source_uri</code> (required)	Acquisition location retained as provenance. string; maxLength=4096; format=uri
<code>acquired_at</code> (required)	Time the bytes were acquired. string; maxLength=40; format=date-time; pattern enforced (see schema)
<code>acquisition_ref</code> (required)	Exact acquisition report when available; local legacy inputs may have no such history. anyOf: allOf: RecordRef; constrained object; null
<code>acquisition_method</code> (required)	How these bytes were obtained. one of ARXIV_SOURCE, ARXIV_PDF, LOCAL_IMPORT, OTHER_EXACT_SOURCE
<code>units</code> (required)	Complete supplied source inventory. array of SourceUnit; minItems=1; maxItems=10000
<code>archive</code> (required)	Original source archive if available. anyOf: ArtifactRef; null
<code>missing_material</code> (required)	Source material known to be absent. array of string; maxItems=10000
<code>rights_status</code> (required)	Recorded availability and reuse-rights boundary. one of KNOWN, UNRESOLVED, RESTRICTED
<code>rights_note</code> (required)	Public source and redistribution conditions. string; minLength=1; maxLength=65536

A.3.4 SourceSpan

Exact source occurrence used to attribute a proposition or condition.

Schema: `source-span.schema.json`. **Roadmap:** P2.2, P2.4.

Field	Meaning and local constraints
<code>source_ref</code> (required)	Exact source snapshot containing this occurrence. allOf: RecordRef; constrained object
<code>artifact</code> (required)	Raw source bytes containing the occurrence. ArtifactRef
<code>locator_kind</code> (required)	Exact source occurrence representation. one of RAW_TEXT_BYTES, PDF_REGION, TRANSFORMED_TEXT
<code>byte_start</code> (required)	Inclusive zero-based byte offset; null for a PDF page region. anyOf: integer; null
<code>byte_end</code> (required)	Exclusive byte offset, null for a PDF page region. anyOf: integer; null
<code>span_sha256</code> (required)	Exact selected text-byte digest; null for a PDF page region. anyOf: string; null
<code>pdf_region</code> (required)	Page and box for a PDF occurrence. anyOf: PdfRegion; null

Field	Meaning and local constraints (continued)
transformation_ref (required)	Exact mapping when using derived text. anyOf: allOf: RecordRef; constrained object; null
locator (required)	Human-readable page, line or label. string; minLength=1; maxLength=4096
activity (required)	Occurrence participation in the chosen source rendering. one of ACTIVE, DORMANT, UNKNOWN

A.3.5 SourceTransformation

An auditable mapping from raw material to a derived representation.

Schema: source-transformation.schema.json. **Roadmap:** P2.2.

Field	Meaning and local constraints
source_ref (required)	Exact source input. allOf: RecordRef; constrained object
inputs (required)	Unmodified input bytes. array of ArtifactRef; minItems=1; maxItems=10000
outputs (required)	Derived bytes with distinct identities. array of ArtifactRef; minItems=1; maxItems=10000
method (required)	Extraction, expansion or rendering procedure. string; minLength=1; maxLength=65536
environment (required)	Fixed tool and configuration identity. string; minLength=1; maxLength=65536
mapping_artifact (required)	Explicit source-to-output location map. ArtifactRef
limitations (required)	Unresolved rendering or decoding ambiguity. array of string; maxItems=10000

A.3.6 PaperStructure

Source containment and activity, distinct from mathematical dependence.

Schema: paper-structure.schema.json. **Roadmap:** P2.2.

Field	Meaning and local constraints
source_ref (required)	Exact source inventory. allOf: RecordRef; constrained object
main_unit_id (required)	Explicit chosen document entry point. string; maxLength=512; pattern enforced (see schema)
edges (required)	Document structure relations. array of StructureEdge; maxItems=10000
unclassified_unit_ids (required)	Units whose activity or role could not be established. array of string; maxItems=10000
blockers (required)	Parsing or source-activity limitations. array of string; maxItems=10000

A.3.7 AgentizationPlan

Query-independent plan fixed before detailed discovery.

Schema: agentization-plan.schema.json. **Roadmap:** P1.3, P2.3.

Field	Meaning and local constraints
<code>source_ref</code> (required)	Exact planned paper source. allOf: RecordRef; constrained object
<code>profile_ref</code> (required)	Exact work requirements. allOf: RecordRef; constrained object
<code>baseline_ref</code> (required)	Prior knowledge selected for later comparison. anyOf: allOf: RecordRef; constrained object; null
<code>source_unit_ids</code> (required)	Fixed source denominator for discovery. array of string; minItems=1; maxItems=10000
<code>max_steps</code> (required)	Maximum bounded work steps. integer; minimum=1; maximum=9007199254740991
<code>max_seconds</code> (required)	Maximum wall-clock work budget. integer; minimum=1; maximum=9007199254740991
<code>max_cost_units</code> (required)	Declared integer resource budget. integer; minimum=0; maximum=9007199254740991
<code>no_progress_limit</code> (required)	Consecutive unproductive attempts before deferral. integer; minimum=1; maximum=9007199254740991
<code>trigger_note</code> (required)	Why work began; it does not define canonical scope. string; minLength=1; maxLength=65536

A.3.8 InventoryDiscovery

All source units classified before an independent scope decision.

Schema: inventory-discovery.schema.json. **Roadmap:** P2.3, P2.6.

Field	Meaning and local constraints
<code>plan_ref</code> (required)	Exact discovery plan. allOf: RecordRef; constrained object
<code>structure_ref</code> (required)	Source structure used for discovery. allOf: RecordRef; constrained object
<code>classified_unit_ids</code> (required)	Units assigned an explicit role. array of string; maxItems=10000
<code>unclassified_unit_ids</code> (required)	Units still unresolved but retained in the denominator. array of string; maxItems=10000
<code>obligations</code> (required)	Proposed complete work denominator. array of Obligation; minItems=1; maxItems=10000
<code>claim_refs</code> (required)	Candidate or attributed source propositions. array of allOf: RecordRef; constrained object; maxItems=10000
<code>coverage_rationale</code> (required)	Public account of completeness and exclusions. string; minLength=1; maxLength=65536

A.3.9 ScopeFreezeDecision

Independent acceptance or blocking of an exact discovery.

Schema: scope-decision.schema.json. **Roadmap:** P2.6.

Field	Meaning and local constraints
<code>discovery_ref</code> (required)	Exact discovery under review. allOf: RecordRef; constrained object
<code>review</code> (required)	Independent review boundary. ReviewContext
<code>decision</code> (required)	Freeze decision, not scientific acceptance. one of BLOCK, ACCEPT
<code>reason</code> (required)	Public decision rationale. string; minLength=1; maxLength=65536

A.3.10 FrozenInventoryScope

Immutable work denominator after an independent freeze decision.

Schema: frozen-scope.schema.json. **Roadmap:** P1.3, P2.5.

Field	Meaning and local constraints
<code>plan_ref</code> (required)	Exact governing plan. allOf: RecordRef; constrained object
<code>discovery_ref</code> (required)	Exact inventoried obligations. allOf: RecordRef; constrained object
<code>decision_ref</code> (required)	Accepting independent freeze decision. allOf: RecordRef; constrained object
<code>obligations</code> (required)	Exact obligations retained for total accounting. array of Obligation; minItems=1; maxItems=10000
<code>predecessor_ref</code> (required)	Previous immutable scope, if revised. anyOf: allOf: RecordRef; constrained object; null
<code>change_reason</code> (required)	Genesis reason or explicit scope extension/correction rationale. string; minLength=1; maxLength=65536
<code>removed_obligation_dispositions</code> (required)	Explicit reviewed accounting for every removed prior obligation. array of RecordRef; maxItems=10000

A.3.11 ObligationDisposition

Current disposition of one frozen obligation, retaining all attempts.

Schema: obligation-disposition.schema.json. **Roadmap:** P1.3.

Field	Meaning and local constraints
<code>scope_ref</code> (required)	Exact frozen denominator. allOf: RecordRef; constrained object
<code>obligation_id</code> (required)	Exact obligation being accounted for. string; maxLength=512; pattern enforced (see schema)
<code>outcome</code> (required)	Execution accounting only. one of BLOCKED, DEFERRED, FAILED, COMPLETED, NOT_APPLICABLE

Field	Meaning and local constraints (continued)
result_refs (required)	Actual produced records, not invented placeholders. array of RecordRef; maxItems=10000
attempt_refs (required)	Exact work attempts. array of allOf: RecordRef; constrained object; maxItems=10000
reason (required)	Outcome explanation and remaining boundary. string; minLength=1; maxLength=65536
previous_ref (required)	Prior immutable disposition. anyOf: allOf: RecordRef; constrained object; null

A.3.12 ScientificClaim

Source-faithful proposition preserving conditions, attribution and scientific scope.

Schema: scientific-claim.schema.json. **Roadmap:** P2.4, P2.5.

Field	Meaning and local constraints
statement (required)	Complete independently revisable source proposition. string; minLength=1; maxLength=65536
source_span_refs (required)	Exact source occurrences supporting every material part. array of allOf: RecordRef; constrained object; minItems=1; maxItems=10000
components (required)	Independently addressable statement parts. array of Component; minItems=1; maxItems=10000
conditions (required)	All truth-relevant source conditions. array of Condition; maxItems=10000
modality (required)	Source strength and modality. one of ASSERTED, CONJECTURED, OBSERVED, APPROXIMATE, CONDITIONAL, INTERPRETIVE
attribution (required)	Who asserts which part. string; minLength=1; maxLength=65536
system (required)	Model, physical system or mathematical domain. string; minLength=1; maxLength=65536
comparison_baseline (required)	Source comparison baseline when asserted. string; maxLength=65536

A.3.13 DerivedClaim

New proposition with explicit transformation and no fabricated source attribution.

Schema: derived-claim.schema.json. **Roadmap:** P2.5, P4.5.

Field	Meaning and local constraints
statement (required)	Complete new proposition. string; minLength=1; maxLength=65536
origin (required)	Why a separate proposition exists. one of SYSTEM_DERIVATION, CONDITIONALIZATION, REPAIR, INDEPENDENT_PROPOSITION
parent_refs (required)	Exact propositions from which it was derived or distinguished. array of allOf: RecordRef; constrained object; maxItems=10000

Field	Meaning and local constraints (continued)
added_conditions (required)	Conditions absent from the original proposition. array of Condition; maxItems=10000
relation (required)	Declared relation requiring separate assessment. one of DERIVES, SPECIALIZES, CORRECTS, ALTERNATIVE, INDEPENDENT
rationale (required)	Public reason for the transformation. string; minLength=1; maxLength=65536

A.3.14 ClaimComponentMap

Complete mapped-or-residual accounting without scientific promotion.

Schema: claim-component-map.schema.json. **Roadmap:** P2.4.

Field	Meaning and local constraints
claim_ref (required)	Exact original claim. allOf: RecordRef; constrained object
mappings (required)	Exactly one accounting row for each source component. array of ComponentMapping; minItems=1; maxItems=10000
review (required)	Source-to-representation review. ReviewContext
coverage (required)	Mathematical/semantic accounting coverage, not truth. one of PARTIAL, COMPLETE

A.3.15 SemanticContext

Scientific meaning, approximation and observable retained outside formal proof.

Schema: semantic-context.schema.json. **Roadmap:** P4.6.

Field	Meaning and local constraints
claim_ref (required)	Exact interpreted proposition. allOf: RecordRef; constrained object
physical_system (required)	Intended physical or modeled system. string; minLength=1; maxLength=65536
object_mapping (required)	Mathematical-to-scientific object correspondence. array of ParameterComparison; minItems=1; maxItems=10000
assumptions (required)	Scientific model assumptions. array of Condition; maxItems=10000
approximation_regime (required)	Range of validity and approximation conditions. string; minLength=1; maxLength=65536
error_bound (required)	Source-supported uncertainty or explicit unknown wording. string; maxLength=65536
observables (required)	Operational definitions of observed quantities. array of string; maxItems=10000
limitations (required)	Unresolved scientific meaning or applicability. array of string; maxItems=10000

A.3.16 MathematicalDefinition

Exact definition with carrier, convention and provenance.

Schema: definition.schema.json. **Roadmap:** P1.4, P3.3.

Field	Meaning and local constraints
name (required)	Definition name. string; minLength=1; maxLength=4096
statement (required)	Complete mathematical definition. string; minLength=1; maxLength=65536
objects (required)	Defined objects and carriers. array of MathObject; minItems=1; maxItems=10000
source_span_refs (required)	Source anchors; empty only for explicit system definitions. array of allOf: RecordRef; constrained object; maxItems=10000
origin (required)	Definition attribution. one of SOURCE, IMPORTED, SYSTEM

A.3.17 MathClaimIR

Exact mathematical target without mutable verification status.

Schema: math-claim.schema.json. **Roadmap:** P1.4, P4.1.

Field	Meaning and local constraints
claim_ref (required)	Exact source or independently derived proposition. allOf: RecordRef; constrained object
component_ids (required)	Mathematical components represented by this target. array of string; maxItems=10000
objects (required)	Carriers, units and conventions. array of MathObject; minItems=1; maxItems=10000
quantifiers (required)	Ordered quantifiers. array of Quantifier; maxItems=10000
assumptions (required)	Complete explicit mathematical premises. array of Condition; maxItems=10000
conclusion (required)	Exact mathematical conclusion. string; minLength=1; maxLength=65536
exactness (required)	Strength of the target. one of EXACT, APPROXIMATE, ASYMPTOTIC
approximation_error (required)	Error and regime when nonexact. string; maxLength=65536
definition_refs (required)	Exact mathematical definitions. array of allOf: RecordRef; constrained object; maxItems=10000
semantic_refs (required)	Scientific residual meaning. array of allOf: RecordRef; constrained object; maxItems=10000
normalization (required)	Symbol and convention normalization with no scope change. string; minLength=1; maxLength=65536

A.3.18 AssumptionContext

Local assumptions and their nesting; discharge is an explicit inference.

Schema: `assumption-context.schema.json`. **Roadmap:** P3.3, P3.6.

Field	Meaning and local constraints
<code>parent_ref</code> (required)	Enclosing context, never a cyclic parent. anyOf: allOf: RecordRef; constrained object; null
<code>assumptions</code> (required)	Assumptions introduced in this context. array of Condition; minItems=1; maxItems=10000
<code>introduction_reason</code> (required)	Where and why these local premises enter. string; minLength=1; maxLength=65536
<code>allowed_use</code> (required)	Boundaries of valid use. string; minLength=1; maxLength=65536

A.3.19 ArgumentNode

One attributed statement in a mathematical argument.

Schema: `argument-node.schema.json`. **Roadmap:** P3.2, P3.3.

Field	Meaning and local constraints
<code>statement</code> (required)	Complete node proposition. string; minLength=1; maxLength=65536
<code>kind</code> (required)	Mathematical role. one of CLAIM, DEFINITION, ASSUMPTION, LEMMA, CONSTRUCTION, CONCLUSION
<code>origin</code> (required)	Attribution of this argument content. one of SOURCE, RECONSTRUCTED, REPAIRED, ALTERNATIVE, IMPORTED
<code>claim_refs</code> (required)	Exact claim correspondence. array of allOf: RecordRef; constrained object; maxItems=10000
<code>context_ref</code> (required)	Local assumption scope. anyOf: allOf: RecordRef; constrained object; null
<code>definition_refs</code> (required)	Definitions used by the node. array of allOf: RecordRef; constrained object; maxItems=10000
<code>source_span_refs</code> (required)	Exact source support where applicable. array of allOf: RecordRef; constrained object; maxItems=10000

A.3.20 InferenceStep

Joint-premise inference distinct from document containment.

Schema: `inference-step.schema.json`. **Roadmap:** P1.4, P3.3.

Field	Meaning and local constraints
<code>premise_refs</code> (required)	Jointly required premises, not separate entailments. array of allOf: RecordRef; constrained object; maxItems=10000
<code>conclusion_ref</code> (required)	Exact concluded node. allOf: RecordRef; constrained object

Field	Meaning and local constraints (continued)
rule (required)	Explicit inference rule. one of DIRECT, MODUS_PONENS, DEFINITIONAL, CONTRADICTION, INDUCTION, CASE_SPLIT, ASSUMPTION_DISCHARGE, EXTERNAL_RESULT
justification (required)	Public mathematical justification. string; minLength=1; maxLength=65536
context_ref (required)	Scope in which the inference is valid. anyOf: allOf: RecordRef; constrained object; null
discharged_context_refs (required)	Exact contexts discharged by this step. array of allOf: RecordRef; constrained object; maxItems=10000
rule_evidence_refs (required)	Witnesses for rule-specific conditions. array of RecordRef; maxItems=10000

A.3.21 ProofPlan

One independent route supporting a conclusion.

Schema: proof-plan.schema.json. **Roadmap:** P3.3, P4.5.

Field	Meaning and local constraints
scope_ref (required)	Exact frozen source of required proof obligations. allOf: RecordRef; constrained object
conclusion_ref (required)	Exact target conclusion. allOf: RecordRef; constrained object
route (required)	Attribution of the proof method. one of ORIGINAL, RECONSTRUCTED, REPAIRED, ALTERNATIVE
node_refs (required)	Nodes participating in this route. array of allOf: RecordRef; constrained object; minItems=1; maxItems=10000
inference_refs (required)	Exact joint-premise steps. array of allOf: RecordRef; constrained object; maxItems=10000
required_obligation_ids (required)	Frozen proof obligations that cannot disappear on deletion. array of string; maxItems=10000
open_obligation_ids (required)	Obligations still not discharged. array of string; maxItems=10000

A.3.22 DependencyBinding

Exact use of a premise, definition or external result.

Schema: dependency-binding.schema.json. **Roadmap:** P3.6, P5.4.

Field	Meaning and local constraints
dependent_ref (required)	Exact object that needs the dependency. RecordRef
prerequisite_ref (required)	Exact required input. RecordRef
kind (required)	Type governing review propagation. one of MATHEMATICAL, DEFINITION, SCOPE, FORMAL_ENVIRONMENT, SEMANTIC, EMPIRICAL, COMPUTATIONAL, VALIDATION

Field	Meaning and local constraints (continued)
affected_axes (required)	Axes affected by this dependency. array of one of <code>source_fidelity</code> , <code>mathematical_correctness</code> , <code>formal_alignment</code> , <code>semantic_applicability</code> , <code>empirical_support</code> , <code>computational_reproducibility</code> ; <code>minItems=1</code> ; <code>maxItems=10000</code>
proof_plan_ref (required)	Proof route requiring this input. <code>anyOf</code> : <code>allOf</code> : <code>RecordRef</code> ; constrained object; null
comparison (required)	Exact assumption and object compatibility. array of <code>ParameterComparison</code> ; <code>minItems=1</code> ; <code>maxItems=10000</code>
reuse_decision_ref (required)	Separate authorization for the intended use. <code>anyOf</code> : <code>allOf</code> : <code>RecordRef</code> ; constrained object; null

A.3.23 Challenge

Precisely targeted objection; a response does not resolve it.

Schema: `challenge.schema.json`. **Roadmap:** P3.5.

Field	Meaning and local constraints
target_ref (required)	Exact questioned node, inference or binding. <code>RecordRef</code>
category (required)	Question category. one of <code>INFERENCE</code> , <code>SCOPE</code> , <code>DEFINITION</code> , <code>PROVENANCE</code> , <code>COUNTEREXAMPLE</code> , <code>ALIGNMENT</code> , <code>DEPENDENCY</code> , <code>AUTHORITY</code>
severity (required)	Review priority, not probability of error. one of <code>CRITICAL</code> , <code>MAJOR</code> , <code>MINOR</code> , <code>NOTE</code>
statement (required)	Public concrete objection. string; <code>minLength=1</code> ; <code>maxLength=65536</code>
evidence_refs (required)	Supporting evidence for the objection. array of <code>RecordRef</code> ; <code>maxItems=10000</code>

A.3.24 ChallengeDisposition

Independent disposition of an exact objection and response.

Schema: `challenge-disposition.schema.json`. **Roadmap:** P3.5, P3.7.

Field	Meaning and local constraints
challenge_ref (required)	Exact original objection. <code>allOf</code> : <code>RecordRef</code> ; constrained object
response (required)	Public response without private model reasoning. string; <code>minLength=1</code> ; <code>maxLength=65536</code>
response_refs (required)	Revised or additional mathematical evidence. array of <code>RecordRef</code> ; <code>maxItems=10000</code>
review (required)	Independent review of whether the objection is answered. <code>ReviewContext</code>
outcome (required)	Disposition retaining the original challenge. one of <code>OPEN</code> , <code>RESOLVED</code> , <code>WITHDRAWN</code> , <code>SUPERSEDED</code> , <code>DISPUTED</code>
replacement_ref (required)	New objection if superseded. <code>anyOf</code> : <code>allOf</code> : <code>RecordRef</code> ; constrained object; null

Field	Meaning and local constraints (continued)
reason (required)	Why the disposition applies to this exact revision. string; minLength=1; maxLength=65536

A.3.25 ArgumentSnapshot

Full fixed argument, not merely the upstream graph projection.

Schema: `argument-snapshot.schema.json`. **Roadmap:** P3.2.

Field	Meaning and local constraints
scope_ref (required)	Exact query-independent argument work scope. allOf: RecordRef; constrained object
source_claim_refs (required)	Exact argument targets. array of allOf: RecordRef; constrained object; minItems=1; maxItems=10000
math_refs (required)	Fixed mathematical targets associated with the argument. array of allOf: RecordRef; constrained object; maxItems=10000
node_refs (required)	Full node inventory. array of allOf: RecordRef; constrained object; minItems=1; maxItems=10000
inference_refs (required)	Joint-premise inferences. array of allOf: RecordRef; constrained object; maxItems=10000
context_refs (required)	All local assumption scopes. array of allOf: RecordRef; constrained object; maxItems=10000
proof_plan_refs (required)	Distinct proof routes. array of allOf: RecordRef; constrained object; minItems=1; maxItems=10000
dependency_refs (required)	Full reference and definition dependencies. array of allOf: RecordRef; constrained object; maxItems=10000
challenge_refs (required)	All objections including archived paths. array of allOf: RecordRef; constrained object; maxItems=10000
challenge_disposition_refs (required)	Complete known objection history. array of allOf: RecordRef; constrained object; maxItems=10000
upstream_import_ref (required)	Exact preserved engine input; null for native arguments. anyOf: allOf: RecordRef; constrained object; null
missing_items (required)	Explicit incompleteness of this snapshot. array of string; maxItems=10000

A.3.26 ArgumentReview

Independent argument assessment with explicit open obligations.

Schema: `argument-review.schema.json`. **Roadmap:** P3.8.

Field	Meaning and local constraints
snapshot_ref (required)	Exact complete review context. allOf: RecordRef; constrained object
proof_plan_refs (required)	Routes actually reviewed. array of allOf: RecordRef; constrained object; minItems=1; maxItems=10000
review (required)	Actual review identity and procedure. ReviewContext

Field	Meaning and local constraints (continued)
outcome (required)	Scoped argument result, not kernel proof. one of INCONCLUSIVE, CONDITIONAL, SUPPORTED, COUNTEREVIDENCE
open_obligation_ids (required)	Remaining load-bearing obligations. array of string; maxItems=10000
open_challenge_refs (required)	Unresolved relevant objections. array of allOf: RecordRef; constrained object; maxItems=10000
conditions (required)	Conditions retained with the conclusion. array of Condition; maxItems=10000

A.3.27 UpstreamImport

Preserved engine report and independently checked import boundary.

Schema: upstream-import.schema.json. **Roadmap:** P3.1, P3.2, P3.8.

Field	Meaning and local constraints
engine (required)	Upstream engine identity. constant vibefeld
commit (required)	Exact source commit. string; maxLength=64; pattern enforced (see schema)
protocol_version (required)	Exact supported export protocol. string; minLength=1; maxLength=4096
adapter_version (required)	Exact AgtXiv importer version. string; minLength=1; maxLength=4096
ledger (required)	Raw event ledger, or null when unavailable. anyOf: ArtifactRef; null
graph (required)	Raw graph projection, or null when unavailable. anyOf: ArtifactRef; null
context_artifacts (required)	Available scope, definitions, externals, identity and full objections. array of ArtifactRef; maxItems=10000
identity_map (required)	Exact identity mapping, null if unavailable. anyOf: ArtifactRef; null
reported_states (required)	Original reported states, null if unavailable. anyOf: ArtifactRef; null
import_checks (required)	Explicit checks that actually ran. array of string; minItems=1; maxItems=10000
outcome (required)	Import result, never scientific support. one of INCOMPLETE, REJECTED, STRUCTURALLY_IMPORTED
missing_items (required)	Required upstream information absent. array of string; maxItems=10000

A.3.28 WorkAttempt

Bounded attributable execution with measured resource use.

Schema: `work-attempt.schema.json`. **Roadmap:** P3.5, P8.3.

Field	Meaning and local constraints
<code>plan_ref</code> (required)	Governing work budget. allOf: RecordRef; constrained object
<code>operation</code> (required)	Executed operation. string; minLength=1; maxLength=4096
<code>target_refs</code> (required)	Exact inputs or targets. array of RecordRef; minItems=1; maxItems=10000
<code>started_at</code> (required)	Execution start time. string; maxLength=40; format=date-time; pattern enforced (see schema)
<code>finished_at</code> (required)	Execution stop time. string; maxLength=40; format=date-time; pattern enforced (see schema)
<code>outcome</code> (required)	Execution outcome without scientific promotion. one of FAILED, BLOCKED, DEFERRED, SUCCEEDED
<code>output_refs</code> (required)	Already-created outputs only; records referring back to this attempt are listed by a later completion receipt. array of RecordRef; maxItems=10000
<code>logs</code> (required)	Preserved public execution logs. array of ArtifactRef; maxItems=10000
<code>cost_units</code> (required)	Observed resource units. integer; minimum=0; maximum=9007199254740991
<code>progress_observed</code> (required)	Whether a new result or actionable diagnosis was obtained. boolean

A.3.29 FormalEnvironment

Exact toolchain, packages, build inputs and permitted trust policy.

Schema: `formal-environment.schema.json`. **Roadmap:** P4.2, P4.3.

Field	Meaning and local constraints
<code>prover</code> (required)	Formal checker family. constant Lean4
<code>toolchain</code> (required)	Exact Lean toolchain identifier. string; minLength=1; maxLength=4096
<code>package_manifest</code> (required)	Exact package commit lock. ArtifactRef
<code>source_artifacts</code> (required)	All supplied formal build inputs. array of ArtifactRef; minItems=1; maxItems=10000
<code>allowed_axioms</code> (required)	Explicit permitted axiom names. array of string; maxItems=10000
<code>allowed_trust_mechanisms</code> (required)	Explicit additional accepted checker mechanisms. array of string; maxItems=10000
<code>command</code> (required)	Argument-vector command, never an interpolated shell string. array of string; minItems=1; maxItems=10000
<code>network_access</code> (required)	Declared execution network boundary. one of DISABLED, CONTROLLED_ACQUISITION

A.3.30 FormalizationPacket

Fixed mathematical target, argument route, semantics and environment.

Schema: formalization-packet.schema.json. **Roadmap:** P4.1.

Field	Meaning and local constraints
<code>scope_ref</code> (required)	Exact source of handoff obligations. allOf: RecordRef; constrained object
<code>math_ref</code> (required)	Exact mathematical target. allOf: RecordRef; constrained object
<code>argument_ref</code> (required)	Exact complete argument snapshot. allOf: RecordRef; constrained object
<code>proof_plan_ref</code> (required)	Selected proof route. allOf: RecordRef; constrained object
<code>source_claim_refs</code> (required)	Exact original or derived targets. array of allOf: RecordRef; constrained object; minItems=1; maxItems=10000
<code>semantic_refs</code> (required)	Residual scientific meaning. array of allOf: RecordRef; constrained object; maxItems=10000
<code>environment_ref</code> (required)	Fixed checker environment. allOf: RecordRef; constrained object
<code>expected_declarations</code> (required)	Required fully qualified output declarations. array of string; minItems=1; maxItems=10000
<code>open_obligation_ids</code> (required)	Explicit unproved obligations at handoff. array of string; maxItems=10000
<code>mode</code> (required)	Work purpose; neither value grants authority. one of EXPLORATORY, RELEASE_CANDIDATE

A.3.31 FormalizationAttempt

Generated formal files and diagnostics, distinct from independent checking.

Schema: formalization-attempt.schema.json. **Roadmap:** P4.1, P4.3.

Field	Meaning and local constraints
<code>packet_ref</code> (required)	Exact handoff. allOf: RecordRef; constrained object
<code>attempt_ref</code> (required)	Exact generation run. allOf: RecordRef; constrained object
<code>outcome</code> (required)	File-generation result only. one of FAILED, PARTIAL, GENERATED
<code>artifacts</code> (required)	Actual generated source files. array of ArtifactRef; maxItems=10000
<code>diagnostics</code> (required)	Public errors and unresolved work. array of string; maxItems=10000

A.3.32 FormalCheckRecord

Independent target-level kernel and transitive axiom evidence.

Schema: formal-check.schema.json. **Roadmap:** P4.3.

Field	Meaning and local constraints
<code>packet_ref</code> (required)	Exact target handoff. allOf: RecordRef; constrained object
<code>attempt_ref</code> (required)	Files actually checked. allOf: RecordRef; constrained object
<code>environment_ref</code> (required)	Exact checker environment. allOf: RecordRef; constrained object
<code>review</code> (required)	Independence from formal source generation. ReviewContext
<code>outcome</code> (required)	Scoped observed checker result. one of FAILED, BLOCKED, KERNEL_CHECKED
<code>built_declarations</code> (required)	Declarations actually inspected in the environment. array of FormalDeclaration; maxItems=10000
<code>placeholder_findings</code> (required)	Detected unresolved placeholders or forbidden mechanisms. array of string; maxItems=10000
<code>logs</code> (required)	Independent checker and axiom-audit logs. array of ArtifactRef; minItems=1; maxItems=10000
<code>exit_code</code> (required)	Actual process exit code represented as a nonnegative integer. integer; minimum=0; maximum=9007199254740991

A.3.33 BacktranslationRecord

Formal statement interpretation with disclosed input visibility.

Schema: backtranslation.schema.json. **Roadmap:** P4.4.

Field	Meaning and local constraints
<code>packet_ref</code> (required)	Exact target packet identity, not permission to read source text. allOf: RecordRef; constrained object
<code>formal_artifacts</code> (required)	Formal statements and required definitions actually visible. array of ArtifactRef; minItems=1; maxItems=10000
<code>visible_record_refs</code> (required)	Exact records visible to the interpreter. array of RecordRef; maxItems=10000
<code>source_blind</code> (required)	Declared source blindness requiring execution evidence. boolean
<code>interpretation</code> (required)	What the formal statement means including assumptions. string; minLength=1; maxLength=65536
<code>conditions</code> (required)	Conditions found in the declaration. array of Condition; maxItems=10000
<code>visibility_evidence</code> (required)	Independent input-visibility evidence. array of ArtifactRef; minItems=1; maxItems=10000

A.3.34 AlignmentAssessment

One explicit semantic comparison with exact endpoints and independent review.

Schema: alignment-assessment.schema.json. **Roadmap:** P4.4.

Field	Meaning and local constraints
<code>comparison_kind</code> (required)	Which alignment boundary was checked. one of SOURCE_TO_ARGUMENT, SOURCE_TO_MATH, MATH_TO_FORMAL, SOURCE_TO_FORMAL
<code>source_refs</code> (required)	Exact source-side objects. array of RecordRef; minItems=1; maxItems=10000
<code>target_refs</code> (required)	Exact target-side objects. array of RecordRef; minItems=1; maxItems=10000
<code>review</code> (required)	Independent semantic review. ReviewContext
<code>comparisons</code> (required)	Quantifiers, types, assumptions, meaning and strength comparisons. array of ParameterComparison; minItems=1; maxItems=10000
<code>outcome</code> (required)	Scoped alignment only, not proof correctness. one of UNKNOWN, PARTIAL, MISALIGNED, ALIGNED
<code>backtranslation_refs</code> (required)	Independently generated formal interpretations. array of allOf: RecordRef; constrained object; maxItems=10000

A.3.35 EmpiricalEvidence

Observations, uncertainty and methods tied to exact scientific targets.

Schema: empirical-evidence.schema.json. **Roadmap:** P4.7.

Field	Meaning and local constraints
<code>target_refs</code> (required)	Exact scientific assumptions or conclusions examined. array of allOf: RecordRef; constrained object; minItems=1; maxItems=10000
<code>data</code> (required)	Exact observation data. array of ArtifactRef; minItems=1; maxItems=10000
<code>method</code> (required)	Data collection and analysis method. string; minLength=1; maxLength=65536
<code>population_regime</code> (required)	Population, system and applicable regime. string; minLength=1; maxLength=65536
<code>uncertainty</code> (required)	Error model and uncertainty reporting. string; minLength=1; maxLength=65536
<code>limitations</code> (required)	Bias, missingness and generalization limits. array of string; maxItems=10000
<code>role</code> (required)	Evidence importance to the stated conclusion. one of ILLUSTRATIVE, SUPPORTING, LOAD_BEARING

A.3.36 ReproductionRecord

Observed reproduction result under a fixed procedure and comparison rule.

Schema: reproduction-record.schema.json. **Roadmap:** P4.7.

Field	Meaning and local constraints
<code>protocol_ref</code> (required)	Exact comparison and input policy frozen before execution. allOf: RecordRef; constrained object
<code>attempt_ref</code> (required)	Execution whose fixed inputs included the protocol. allOf: RecordRef; constrained object
<code>target_refs</code> (required)	Exact computational claims or artifacts. array of RecordRef; minItems=1; maxItems=10000
<code>code</code> (required)	Exact executed code. array of ArtifactRef; minItems=1; maxItems=10000
<code>inputs</code> (required)	Exact data and parameter inputs. array of ArtifactRef; minItems=1; maxItems=10000
<code>environment</code> (required)	Exact environment and dependency description. ArtifactRef
<code>command</code> (required)	Executed argument-vector command. array of string; minItems=1; maxItems=10000
<code>seeds</code> (required)	Random seeds or an empty list for a deterministic procedure. array of integer; maxItems=10000
<code>comparison_rule</code> (required)	Predetermined tolerance or statistical comparison method. string; minLength=1; maxLength=65536
<code>outcome</code> (required)	Reproduction result, not scientific validity. one of FAILED, BLOCKED, DISAGREEMENT, REPRODUCED
<code>outputs</code> (required)	Actual produced numerical outputs. array of ArtifactRef; maxItems=10000
<code>logs</code> (required)	Exact independent execution evidence. array of ArtifactRef; minItems=1; maxItems=10000

A.3.37 AxisAssessment

Exactly one of the six Charter questions with bounded evidence.

Schema: axis-assessment.schema.json. **Roadmap:** P1.2, P4.8.

Field	Meaning and local constraints
<code>target_refs</code> (required)	Exact objects assessed on this axis. array of RecordRef; minItems=1; maxItems=10000
<code>axis</code> (required)	One independent Charter verification axis. one of source_fidelity, mathematical_correctness, formal_alignment, semantic_applicability, empirical_support, computational_reproducibility
<code>applicability</code> (required)	Whether this question applies to these targets. one of UNDETERMINED, APPLICABLE, NOT_APPLICABLE
<code>result</code> (required)	Evidence conclusion, independent of execution success. one of NOT_ASSESSED, INCONCLUSIVE, PARTIALLY_SUPPORTED, SUPPORTED, COUNTEREVIDENCE
<code>execution</code> (required)	What assessment work actually completed. one of DEFERRED, BLOCKED, FAILED, COMPLETED

Field	Meaning and local constraints (continued)
method (required)	Evidence method and strength. one of SOURCE_REVIEW, ARGUMENT_REVIEW, KERNEL_AND_ALIGNMENT, FORMAL_ALIGNMENT, SEMANTIC_REVIEW, EMPIRICAL_REVIEW, REPRODUCTION_REVIEW, COUNTEREXAMPLE_REVIEW, UNASSESSED
review (required)	Attributable independent assessment boundary. ReviewContext
support_refs (required)	Exact supporting evidence. array of RecordRef; maxItems=10000
counterevidence_refs (required)	Exact relevant contrary evidence. array of RecordRef; maxItems=10000
conditions (required)	Conditions retained with this assessment. array of Condition; maxItems=10000
frontier_refs (required)	Relevant unresolved problems. array of allOf: RecordRef; constrained object; maxItems=10000
rationale (required)	Public scoped conclusion including non-applicability reasons. string; minLength=1; maxLength=65536

A.3.38 StatusView

Rebuildable six-axis view preserving all exact and conflicting assessments.

Schema: status-view.schema.json. **Roadmap:** P4.8, P7.4.

Field	Meaning and local constraints
target_ref (required)	Exact queried subject. RecordRef
assessment_refs (required)	Full selected assessment evidence. array of allOf: RecordRef; constrained object; maxItems=10000
conflicting_assessment_refs (required)	Independently inspectable disagreements. array of allOf: RecordRef; constrained object; maxItems=10000
axes (required)	Exactly the six axes, each appearing once. array of one of source_fidelity, mathematical_correctness, formal_alignment, semantic_applicability, empirical_support, computational_reproducibility; minItems=6; maxItems=6; uniqueItems=True
frontier_refs (required)	Unresolved boundaries. array of allOf: RecordRef; constrained object; maxItems=10000
projection_method (required)	Deterministic view policy, never a truth-score aggregation. string; minLength=1; maxLength=65536

A.3.39 BaselineSnapshot

Fixed prior knowledge and search coverage for a contribution comparison.

Schema: `baseline-snapshot.schema.json`. **Roadmap:** P5.1.

Field	Meaning and local constraints
<code>knowledge_ref</code> (required)	Exact previous knowledge snapshot, if available. anyOf: allOf: RecordRef; constrained object; null
<code>entry_refs</code> (required)	Exact prior entries included in the comparison. array of RecordRef; maxItems=10000
<code>domain</code> (required)	Scientific and mathematical comparison domain. string; minLength=1; maxLength=65536
<code>selection_method</code> (required)	How the prior knowledge was chosen. string; minLength=1; maxLength=65536
<code>search_coverage</code> (required)	Bounded retrieval sources, queries and coverage. string; minLength=1; maxLength=65536
<code>limitations</code> (required)	Known missing knowledge and non-priority implications. array of string; minItems=1; maxItems=10000

A.3.40 RelationAssessment

Witness-backed relation between exact claims or knowledge records.

Schema: `relation-assessment.schema.json`. **Roadmap:** P5.2.

Field	Meaning and local constraints
<code>source_ref</code> (required)	Source endpoint; read source relation target. RecordRef
<code>target_ref</code> (required)	Target endpoint, distinct from the source. RecordRef
<code>relation</code> (required)	Exact typed relation. enum (11 exact values; see schema)
<code>review</code> (required)	Independent relation assessment. ReviewContext
<code>comparisons</code> (required)	Domain, assumption and semantic alignment witnesses. array of ParameterComparison; minItems=1; maxItems=10000
<code>outcome</code> (required)	Relation assessment state, not endpoint truth. one of PROPOSED, BLOCKED, DISPUTED, ACCEPTED, REJECTED
<code>witness_refs</code> (required)	Mathematical and scientific support. array of RecordRef; maxItems=10000

A.3.41 ContributionDelta

Attributable knowledge change relative to one fixed baseline.

Schema: contribution-delta.schema.json. **Roadmap:** P5.3.

Field	Meaning and local constraints
baseline_ref (required)	Exact comparison baseline, not the later transaction snapshot. allOf: RecordRef; constrained object
current_refs (required)	This paper's exact result, method or evidence. array of RecordRef; minItems=1; maxItems=10000
prior_refs (required)	Prior objects compared. array of RecordRef; maxItems=10000
operation (required)	Attributable type of change, not an importance score. enum (13 exact values; see schema)
relation_refs (required)	Assessed comparison relations. array of allOf: RecordRef; constrained object; maxItems=10000
author_declaration (required)	The paper's own novelty statement, attributed separately. string; maxLength=65536
review (required)	Independent contribution evaluation. ReviewContext
outcome (required)	Bounded delta assessment. one of PROPOSED, BLOCKED, DISPUTED, SUPPORTED, REJECTED
qualifications (required)	Baseline coverage and unresolved limitations. array of string; minItems=1; maxItems=10000

A.3.42 ReuseDecision

Permission to reuse an exact result for one specified target and evidence standard.

Schema: reuse-decision.schema.json. **Roadmap:** P5.4.

Field	Meaning and local constraints
source_ref (required)	Exact reusable result. RecordRef
target_ref (required)	Exact proposed target use. RecordRef
profile_ref (required)	Evidence standard required by this use. allOf: RecordRef; constrained object
comparisons (required)	Required object, premise, scope, evidence and environment matching. array of ParameterComparison; minItems=1; maxItems=10000
assessment_refs (required)	Evidence and relation decisions supporting reuse. array of RecordRef; minItems=1; maxItems=10000
review (required)	Independent use-specific review. ReviewContext
outcome (required)	Use permission, not a new theorem proof. one of UNKNOWN, REJECT, CONDITIONAL, ALLOW
conditions (required)	Remaining conditions required at the target. array of Condition; maxItems=10000
conflict_refs (required)	Relevant contradictory or limiting relations. array of allOf: RecordRef; constrained object; maxItems=10000
environment_check_ref (required)	Exact formal import compatibility when relevant. anyOf: allOf: RecordRef; constrained object; null

A.3.43 EnvironmentCompatibility

Observed exact formal import compatibility, never text matching.

Schema: `environment-check.schema.json`. **Roadmap:** P4.2, P5.4.

Field	Meaning and local constraints
<code>source_environment_ref</code> (required)	Original environment. allOf: RecordRef; constrained object
<code>target_environment_ref</code> (required)	Intended use environment. allOf: RecordRef; constrained object
<code>declarations</code> (required)	Exact declarations imported. array of string; minItems=1; maxItems=10000
<code>outcome</code> (required)	Observed compatibility result. one of INCOMPATIBLE, BLOCKED, COMPATIBLE
<code>logs</code> (required)	Actual build/import evidence. array of ArtifactRef; minItems=1; maxItems=10000
<code>assumption_comparison</code> (required)	Assumptions and definition correspondence. array of ParameterComparison; minItems=1; maxItems=10000

A.3.44 FrontierItem

Durable unresolved question and the precise next evidence needed.

Schema: `frontier-item.schema.json`. **Roadmap:** P5.5, P7.3.

Field	Meaning and local constraints
<code>target_refs</code> (required)	Exact affected targets. array of RecordRef; minItems=1; maxItems=10000
<code>kind</code> (required)	Type of unresolved boundary. one of MISSING_SOURCE, UNPROVED_LEMMA, SCOPE_AMBIGUITY, ALIGNMENT_GAP, MODEL_ASSUMPTION, DATA_MISSING, CONFLICT, ENVIRONMENT, RESOURCE_LIMIT
<code>axes</code> (required)	Applicable affected axes. array of one of source_fidelity, mathematical_correctness, formal_alignment, semantic_applicability, empirical_support, computational_reproducibility; minItems=1; maxItems=10000
<code>statement</code> (required)	Concrete unresolved issue. string; minLength=1; maxLength=65536
<code>next_evidence</code> (required)	What could resolve or narrow the issue. string; minLength=1; maxLength=65536
<code>attempt_refs</code> (required)	Exact previous attempts. array of allOf: RecordRef; constrained object; maxItems=10000
<code>state</code> (required)	Frontier lifecycle, preserving history. one of OPEN, BLOCKED, DEFERRED, RESOLVED, SUPERSEDED
<code>resolution_refs</code> (required)	Evidence when resolved or superseded. array of RecordRef; maxItems=10000

A.3.45 PaperAgentManifest

Exact candidate contents fixed before independent paper audit.

Schema: release-manifest.schema.json. **Roadmap:** P6.1.

Field	Meaning and local constraints
<code>source_ref</code> (required)	Exact paper source. allOf: RecordRef; constrained object
<code>scope_ref</code> (required)	Exact fixed work denominator. allOf: RecordRef; constrained object
<code>profile_ref</code> (required)	Exact completion requirements. allOf: RecordRef; constrained object
<code>record_refs</code> (required)	All packaged structured records. array of RecordRef; minItems=1; maxItems=10000
<code>artifacts</code> (required)	Exact byte artifacts. array of ArtifactRef; maxItems=10000
<code>disposition_refs</code> (required)	One current disposition per frozen obligation. array of allOf: RecordRef; constrained object; minItems=1; maxItems=10000
<code>frontier_refs</code> (required)	All relevant unresolved boundaries. array of allOf: RecordRef; constrained object; maxItems=10000
<code>contribution_refs</code> (required)	At least one assessed, provisional or blocked knowledge delta. array of allOf: RecordRef; constrained object; minItems=1; maxItems=10000
<code>non_implications</code> (required)	Explicit boundaries of package completion. array of string; minItems=1; maxItems=10000

A.3.46 ReleaseAudit

Independent paper-level review of the exact candidate and its scope.

Schema: release-audit.schema.json. **Roadmap:** P6.2.

Field	Meaning and local constraints
<code>manifest_ref</code> (required)	Exact fixed candidate under audit. allOf: RecordRef; constrained object
<code>review</code> (required)	Actual independent paper auditor. ReviewContext
<code>ordered_target_refs</code> (required)	Targets audited in dependency order. array of RecordRef; minItems=1; maxItems=10000
<code>assessment_refs</code> (required)	Exact assessments selected without changing them. array of allOf: RecordRef; constrained object; minItems=1; maxItems=10000
<code>frontier_refs</code> (required)	Complete retained frontier. array of allOf: RecordRef; constrained object; maxItems=10000
<code>recommendation</code> (required)	Profile-scoped recommendation, not paper truth. one of DO_NOT_RELEASE, RECOMMEND_PROFILE_RELEASE
<code>findings</code> (required)	Public audit findings and limits. array of string; minItems=1; maxItems=10000

A.3.47 ReleaseCertificate

Mechanical binding of the exact candidate, audit and authority policy.

Schema: `release-certificate.schema.json`. **Roadmap:** P6.3.

Field	Meaning and local constraints
<code>manifest_ref</code> (required)	Exact candidate certified or rejected. allOf: RecordRef; constrained object
<code>audit_ref</code> (required)	Exact independent audit. allOf: RecordRef; constrained object
<code>checks</code> (required)	Mechanical checks actually performed. array of string; minItems=1; maxItems=10000
<code>outcome</code> (required)	Mechanical result only. one of REJECTED, CERTIFIED
<code>failed_checks</code> (required)	Every failed required check. array of string; maxItems=10000

A.3.48 PaperAgentRelease

Immutable aggregate created after manifest, audit and certificate.

Schema: `paper-release.schema.json`. **Roadmap:** P6.1, P6.4.

Field	Meaning and local constraints
<code>manifest_ref</code> (required)	Exact content manifest. allOf: RecordRef; constrained object
<code>audit_ref</code> (required)	Exact paper audit. allOf: RecordRef; constrained object
<code>certificate_ref</code> (required)	Exact successful certificate. allOf: RecordRef; constrained object
<code>release_name</code> (required)	Human-readable publication version. string; minLength=1; maxLength=4096
<code>publication_uri</code> (required)	Actual public release location, null for local unpublished packages. anyOf: string; null

A.3.49 ArchiveReceipt

Actual persistence receipt independent of knowledge admission.

Schema: `archive-receipt.schema.json`. **Roadmap:** P6.4.

Field	Meaning and local constraints
<code>release_ref</code> (required)	Exact package archived. allOf: RecordRef; constrained object
<code>objects</code> (required)	Bytes whose persistence was checked. array of ArtifactRef; minItems=1; maxItems=10000
<code>checkpoint_ref</code> (required)	Independent history anchor if available. anyOf: allOf: RecordRef; constrained object; null
<code>assurance</code> (required)	Actual archive integrity boundary. one of LOCAL_PERSISTENCE, INDEPENDENTLY_ANCHORED

Field	Meaning and local constraints (continued)
location (required)	Actual archive location. string; maxLength=4096; format=uri

A.3.50 AdmissionDecision

Independent disposition of an exact candidate set for one domain snapshot.

Schema: admission-decision.schema.json. **Roadmap:** P6.5.

Field	Meaning and local constraints
release_ref (required)	Exact source package. allOf: RecordRef; constrained object
before_ref (required)	Exact intended transaction input snapshot. allOf: RecordRef; constrained object
review (required)	Independent admission review. ReviewContext
items (required)	Total disjoint dispositions of the proposed candidates. array of AdmissionItem; minItems=1; maxItems=10000
domain (required)	Scientific reuse domain. string; minLength=1; maxLength=65536

A.3.51 KnowledgeSnapshot

Immutable domain knowledge retaining exact records, relationships and conflicts.

Schema: knowledge-snapshot.schema.json. **Roadmap:** P6.6, P6.7.

Field	Meaning and local constraints
domain (required)	Domain and interpretation scope. string; minLength=1; maxLength=65536
predecessor_ref (required)	Previous immutable snapshot; null for genesis. anyOf: allOf: RecordRef; constrained object; null
entry_refs (required)	Admitted source, evidence, theorem or frontier records. array of RecordRef; maxItems=10000
relation_refs (required)	Assessed relations, including conflicts. array of allOf: RecordRef; constrained object; maxItems=10000
release_refs (required)	Exact backing paper packages. array of allOf: RecordRef; constrained object; maxItems=10000
admission_refs (required)	Decisions authorizing additions. array of allOf: RecordRef; constrained object; maxItems=10000

A.3.52 KnowledgeIngestionReceipt

Atomic before/after transaction with a total candidate partition.

Schema: ingestion-receipt.schema.json. **Roadmap:** P6.6.

Field	Meaning and local constraints
before_ref (required)	Exact snapshot compared before mutation. allOf: RecordRef; constrained object
after_ref (required)	Exact resulting snapshot, identical on abort. allOf: RecordRef; constrained object
decision_ref (required)	Exact accepted/rejected/blocked partition. allOf: RecordRef; constrained object
candidate_refs (required)	Complete transaction candidate universe. array of RecordRef; minItems=1; maxItems=10000
accepted_refs (required)	All and only authorized additions. array of RecordRef; maxItems=10000
rejected_refs (required)	Rejected candidates. array of RecordRef; maxItems=10000
blocked_refs (required)	Candidates still blocked. array of RecordRef; maxItems=10000
outcome (required)	Actual atomic transaction result. one of ABORTED, COMMITTED

A.3.53 EventRecord

Append-only state transition with exact actor and previous-event binding.

Schema: event.schema.json. **Roadmap:** P3.4, P6.4.

Field	Meaning and local constraints
stream_id (required)	Exact event stream. string; maxLength=512; pattern enforced (see schema)
sequence (required)	Monotonic contiguous stream position. integer; minimum=1; maximum=9007199254740991
previous_ref (required)	Exact predecessor; null only for first event. anyOf: allOf: RecordRef; constrained object; null
operation (required)	Attributed state transition. string; minLength=1; maxLength=4096
target_refs (required)	Exact affected records. array of RecordRef; maxItems=10000
receipt_artifacts (required)	Actual external execution evidence. array of ArtifactRef; maxItems=10000

A.3.54 EventCheckpoint

Separately held history anchor with explicit assurance limitations.

Schema: `event-checkpoint.schema.json`. **Roadmap:** P6.4.

Field	Meaning and local constraints
<code>event_ref</code> (required)	Exact anchored event. allOf: RecordRef; constrained object
<code>anchor_artifact</code> (required)	Actual checkpoint or signed attestation bytes. ArtifactRef
<code>holder_principal_id</code> (required)	Principal retaining the checkpoint outside the worker. string; maxLength=512; pattern enforced (see schema)
<code>assurance</code> (required)	Observed anchoring assurance, not inferred from hashes. one of LOCAL_COPY, INDEPENDENTLY_HELD, SIGNATURE_VERIFIED
<code>verification_method</code> (required)	How the anchor was independently checked. string; minLength=1; maxLength=65536

A.3.55 ImpactAnalysis

Typed dependency impact without silently rewriting historical assessments.

Schema: `impact-analysis.schema.json`. **Roadmap:** P7.2.

Field	Meaning and local constraints
<code>trigger_refs</code> (required)	New source, evidence, policy or adopted version triggering review. array of RecordRef; minItems=1; maxItems=10000
<code>dependency_refs</code> (required)	Complete graph inputs used in impact traversal. array of allOf: RecordRef; constrained object; maxItems=10000
<code>affected</code> (required)	Precisely affected targets and axes. array of ImpactTarget; maxItems=10000
<code>unaffected_proof_plan_refs</code> (required)	Independent routes explicitly retained. array of allOf: RecordRef; constrained object; maxItems=10000
<code>algorithm</code> (required)	Versioned deterministic traversal. string; minLength=1; maxLength=4096

A.3.56 RevisionRecord

Explicit semantic or representational change with preserved old identity.

Schema: `revision-record.schema.json`. **Roadmap:** P2.5, P7.3.

Field	Meaning and local constraints
<code>old_ref</code> (required)	Exact preceding object. RecordRef
<code>new_ref</code> (required)	Exact replacement or derived object. RecordRef
<code>change_kind</code> (required)	Semantic type of change. one of REPRESENTATION_CORRECTION, NEW_PROPOSITION, SCOPE_EXTENSION, ENVIRONMENT_CHANGE, ASSESSMENT_UPDATE

Field	Meaning and local constraints (continued)
reason (required)	Public difference and why identity was retained or changed. string; minLength=1; maxLength=65536
impact_ref (required)	Exact affected-set analysis. anyOf: allOf: RecordRef; constrained object; null
review (required)	Independent identity and scope review. ReviewContext

A.3.57 LegacyBinding

Exact legacy evidence without fabricated history or stronger V3 status.

Schema: legacy-binding.schema.json. **Roadmap:** P8.1.

Field	Meaning and local constraints
legacy_contract (required)	Original V1/V2 or research contract identity. string; minLength=1; maxLength=4096
legacy_artifact (required)	Unmodified legacy bytes. ArtifactRef
original_identity (required)	Original IDs, revisions and bound authority. string; minLength=1; maxLength=65536
original_status (required)	Original meaning preserved literally with context. string; minLength=1; maxLength=65536
v3_target_ref (required)	Optional separately evaluated V3 object. anyOf: RecordRef; null
missing_information (required)	Missing identity, review, axes or events. array of string; minItems=1; maxItems=10000
non_promotion (required)	No automatic status upgrade. constant LEGACY_STATUS_PRESERVED

A.3.58 QueryReceipt

Read-only answer over exact knowledge with visible relevant conflict.

Schema: query-receipt.schema.json. **Roadmap:** P7.4, P7.6.

Field	Meaning and local constraints
query (required)	User request used for retrieval, not canonical mutation. string; minLength=1; maxLength=65536
knowledge_ref (required)	Exact queried knowledge snapshot. allOf: RecordRef; constrained object
mode (required)	Authority boundary of the response. one of PACKAGE_BACKED, PROVISIONAL
returned_refs (required)	Exact returned records. array of RecordRef; maxItems=10000
relation_refs (required)	Relevant returned relations including opposing endpoints. array of allOf: RecordRef; constrained object; maxItems=10000
assertions (required)	Public text with exact supporting records. array of QueryAssertion; maxItems=10000

Field	Meaning and local constraints (continued)
frontier_refs (required)	Relevant unresolved issues. array of allOf: RecordRef; constrained object; maxItems=10000
work_request_ref (required)	A separately authorized work plan, if triggered. anyOf: allOf: RecordRef; constrained object; null
canonical_mutation (required)	A query cannot change authoritative knowledge. constant FORBIDDEN

A.3.59 EvaluationReport

Measured comparisons under fixed tasks, profiles and resource boundaries.

Schema: evaluation-report.schema.json. **Roadmap:** P8.3, P8.4.

Field	Meaning and local constraints
corpus (required)	Exact fixed evaluation set. ArtifactRef
profile_ref (required)	Identical work standard used for comparison. allOf: RecordRef; constrained object
methods (required)	Compared processing methods. array of string; minItems=1; maxItems=10000
attempt_refs (required)	Actual runs providing measurements. array of allOf: RecordRef; constrained object; minItems=1; maxItems=10000
metrics (required)	Measured outcomes with denominators. array of Metric; minItems=1; maxItems=10000
limitations (required)	Missing controls and uncertainty preventing stronger claims. array of string; minItems=1; maxItems=10000

A.3.60 SourceRequest

A bibliographic acquisition request that exists before any source bytes are available.

Schema: source-request.schema.json. **Roadmap:** P2.1.

Field	Meaning and local constraints
paper_id (required)	Requested exact work identity. string; minLength=1; maxLength=4096
paper_version (required)	Explicit version, never a latest alias. string; minLength=1; maxLength=4096
source_uri (required)	Requested source location. string; maxLength=4096; format=uri
allowed_hosts (required)	Explicit acquisition host allowlist. array of string; minItems=1; maxItems=10000
max_bytes (required)	Maximum response bytes. integer; minimum=1; maximum=9007199254740991
timeout_seconds (required)	Maximum request duration. integer; minimum=1; maximum=9007199254740991

A.3.61 SourceAcquisition

Observed acquisition success or failure, including failures before a snapshot exists.

Schema: source-acquisition.schema.json. **Roadmap:** P2.1.

Field	Meaning and local constraints
<code>request_ref</code> (required)	Exact request and acquisition policy. allOf: RecordRef; constrained object
<code>outcome</code> (required)	Actual acquisition outcome, not source fidelity. one of FAILED, BLOCKED, ACQUIRED
<code>started_at</code> (required)	Actual request start. string; maxLength=40; format=date-time; pattern enforced (see schema)
<code>finished_at</code> (required)	Actual request completion. string; maxLength=40; format=date-time; pattern enforced (see schema)
<code>artifacts</code> (required)	Actual returned source bytes. array of ArtifactRef; maxItems=10000
<code>final_uri</code> (required)	Final observed URL, null if unavailable. anyOf: string; null
<code>transport_evidence</code> (required)	Retained transport evidence. array of ArtifactRef; maxItems=10000
<code>reason</code> (required)	Failure or success conditions and limitations. string; minLength=1; maxLength=65536

A.3.62 EvidenceProtocol

Evidence targets, inputs and comparison rules frozen before execution.

Schema: evidence-protocol.schema.json. **Roadmap:** P4.7, P8.3.

Field	Meaning and local constraints
<code>scope_ref</code> (required)	Exact work obligation denominator. allOf: RecordRef; constrained object
<code>obligation_ids</code> (required)	Evidence work obligations under this protocol. array of string; minItems=1; maxItems=10000
<code>target_refs</code> (required)	Exact claims or evidence targets. array of RecordRef; minItems=1; maxItems=10000
<code>method</code> (required)	Evidence production method. one of COMPUTATION, EMPIRICAL_ANALYSIS, CONTROLLED_COMPARISON
<code>inputs</code> (required)	Data, code and configuration fixed before execution. array of ArtifactRef; minItems=1; maxItems=10000
<code>environment</code> (required)	Frozen execution environment. ArtifactRef
<code>comparison_rule</code> (required)	Predetermined tolerance or statistical decision rule. string; minLength=1; maxLength=65536
<code>required_outcome</code> (required)	Required evidence obligation. one of ACCOUNT_FOR, SUCCESS_REQUIRED

A.3.63 WorkCompletion

Post-execution reference to results that themselves refer to the attempt.

Schema: `work-completion.schema.json`. **Roadmap:** P3.5, P4.3.

Field	Meaning and local constraints
<code>attempt_ref</code> (required)	Exact earlier execution record. allOf: RecordRef; constrained object
<code>result_refs</code> (required)	Later created results referring to that execution. array of RecordRef; minItems=1; maxItems=10000
<code>limitations</code> (required)	Remaining production or evidence limitations. array of string; maxItems=10000

A.3.64 CounterexampleRecord

A concrete counterexample against one exact quantified proposition.

Schema: `counterexample.schema.json`. **Roadmap:** P3.5, P4.8.

Field	Meaning and local constraints
<code>target_ref</code> (required)	Exact proposition challenged. allOf: RecordRef; constrained object
<code>construction</code> (required)	Explicit mathematical instance or scientific observation. string; minLength=1; maxLength=65536
<code>premise_checks</code> (required)	Why the instance satisfies the target premises and domain. array of ParameterComparison; minItems=1; maxItems=10000
<code>violated_conclusion</code> (required)	Exact target conclusion contradicted by the instance. string; minLength=1; maxLength=65536
<code>evidence_refs</code> (required)	Derivation, checked calculation or observational support. array of RecordRef; minItems=1; maxItems=10000
<code>limitations</code> (required)	Conditions still needing independent review. array of string; maxItems=10000